VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

An Autonomous Institute Affiliated to the University of Mumbai

DEPARTMENT OF COMPUTER ENGINEERING

CAPSTONE PROJECT–II

PROJECT REPORT

ResilientAI: Drift Detection and Diagnosis System

Submitted in partial fulfilment of the requirements for the

Bachelor of Engineering (B.E.) Degree in Computer Engineering

Semester VII | Academic Year 2026–27

Submitted by

Rahul Guhagarkar – Class / Roll No.

Nathan Cherian – Class / Roll No.

Alok Pal – Class / Roll No.

Akul Patre – Class / Roll No.

Project Mentor

Prof./Dr. Sanjay Mirchandani, Designation

A. Y. 2026–27

VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

An Autonomous Institute Affiliated to the University of Mumbai

DEPARTMENT OF COMPUTER ENGINEERING

Certificate

This is to certify that **Rahul Guhagarkar, Nathan Cherian, Alok Pal and Akul Patre** of Fourth Year Computer Engineering studying in Vivekanand Education Society's Institute of Technology, affiliated to University of Mumbai have satisfactorily completed the project on **"ResilientAI: Drift Detection and Diagnosis System"** as a part of their coursework of Capstone Project - II for Semester-VII under the guidance of their mentor **Prof./Dr. Sanjay Mirchandani** in the year 2026 –27.

This project report entitled "ResilientAI: Drift Detection and Diagnosis System" by Rahul Guhagarkar, Nathan Cherian, Alok Pal and Akul Patre is approved for the degree of Bachelor of Engineering (B.E.) in Computer Engineering.

Programme Outcomes	Grade
PO1, PO2, PO3, PO4, PO5, PO6, PO7, PO8, PO9, PO10, PO11, PSO1, PSO2	

Date:

_____

Project Guide: Prof./Dr. Sanjay Mirchandani

Project Report Approval

For

B. E. (Computer Engineering)

This project report entitled “ResilientAI: Drift Detection and Diagnosis System” by Rahul Guhagarkar, Nathan Cherian, Alok Pal and Akul Patre is approved for the degree of Bachelor of Engineering (B.E.) in Computer Engineering.

_____

Internal Examiner

_____

External Examiner

_____

Head of the Department

_____

Principal

Date:

Place:

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

_____

(Signature)
Rahul Guhagarkar (Class / Roll No.)

_____

(Signature)
Nathan Cherian (Class / Roll No.)

_____

(Signature)
Alok Pal (Class / Roll No.)

_____

(Signature)
Akul Patre (Class / Roll No.)

Date:

ACKNOWLEDGEMENT

We are thankful to our college, Vivekanand Education Society's Institute of Technology, for considering our project and extending help at all stages needed during our work of collecting information regarding the project.

It gives us immense pleasure to express our deep and sincere gratitude to our project mentor, Prof./Dr. Sanjay Mirchandani, for the kind help and valuable advice during the development of the project synopsis and for the guidance and suggestions that shaped this work at every review.

We are deeply indebted to our Principal Dr. (Mrs.) J. M. Nair and the Head of the Computer Department Dr. (Mrs.) Nupur Giri for giving us this valuable opportunity to do this project.

We express our hearty thanks to them for their assistance, without which it would have been difficult to finish this project report successfully.

We convey our deep sense of gratitude to all teaching and non-teaching staff for their constant encouragement, support and selfless help throughout the project work. It is a great pleasure to acknowledge the help and suggestions we received from the Department of Computer Engineering.

We wish to express our profound thanks to all those who helped us in gathering information about the project. Our families too have provided moral support and encouragement several times.

DEPARTMENT OF COMPUTER ENGINEERING

COURSE OUTCOMES FOR CAPSTONE PROJECT–II

Course Outcome	Description
CO1	Implement and integrate all modules of the proposed system effectively.
CO2	Perform testing, debugging, validation, and performance optimization of the developed solution.
CO3	Analyze project outcomes using appropriate engineering tools and metrics.
CO4	Demonstrate deployment readiness, usability, security, scalability, and sustainability considerations.
CO5	Analyze, interpret, and present research outcomes effectively using appropriate research methodologies, analytical tools, and presentation techniques.
CO6	Present and defend the complete project professionally through demonstrations, presentations, and comprehensive technical documentation.

CO–PO–PSO Mapping Table

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO 2
CO1	2	3	3	–	3	–	–	–	3	–	2	3	3
CO2	2	3	2	3	3	–	–	–	2	–	2	3	2
CO3	2	3	2	3	3	–	–	–	–	2	2	3	2
CO4	2	2	3	2	3	2	3	2	–	–	3	3	3
CO5	2	3	–	3	3	–	–	2	–	3	2	2	2
CO6	–	–	–	–	2	–	–	2	3	3	2	2	2

Mapping scale: 3 = High correlation; 2 = Moderate correlation; 1 = Low correlation; – = No direct correlation.

The table below shows where this report gives the evidence for each course outcome of Capstone Project II.

CO	How the project meets it	Evidence
CO1	Detection, accuracy estimation, localization, drift type diagnosis, root cause analysis, reliability scoring and remediation triage are implemented as separate modules and joined in one monitoring pipeline with a dashboard.	Chapters 4 and 5
CO2	113 automated tests, statistical checks of the false alarm rate, a benchmark with known drift types and a full rerun after a code review found two errors.	Chapter 6
CO3	Correct-type rate with Wilson intervals, paired permutation tests with Holm correction, mean absolute error of the accuracy estimates, area under the ROC curve and run times.	Sections 7.2 and 7.4

CO	How the project meets it	Evidence
CO4	Local deployment through a Streamlit dashboard, a stated error rate for every decision, measured run times, privacy of the data and a sustainability assessment.	Sections 3.3, 7.1 and 7.7
CO5	Literature and patent survey, hypothesis-testing methodology, a held-out benchmark design and a research paper prepared for publication.	Chapter 2, Section 7.4, Appendix I
CO6	This report, the dashboard demonstration and the project presentation.	This report, Appendix I

TABLE OF CONTENTS

Title.....Page No.

ABSTRACT.....xvi

CHAPTER 1: INTRODUCTION.....1

1.1 Introduction.....1

1.2 Motivation.....2

1.3 Problem Definition.....3

1.4 Existing Systems.....3

1.5 Lacunae in Existing Systems.....4

1.6 Relevance of the Project.....5

1.7 Alignment with the United Nations Sustainable Development Goals (SDGs).....5

CHAPTER 2: LITERATURE SURVEY.....7

2.1 Brief Overview of the Literature Survey.....7

2.2 Related Works / Research Papers Referred.....7

2.3 Patent Search.....9

2.4 Inference Drawn.....10

2.5 Comparison with Existing Systems.....11

2.6 Feasibility Analysis.....12

2.6.1 Technical Feasibility.....12

2.6.2 Economic and Financial Feasibility.....12

2.6.3 Cost-effectiveness Analysis.....13

2.6.4 Operational Feasibility and Risk Analysis.....13

2.6.5 Environmental and Sustainability Feasibility.....14

2.6.6 Legal, Ethical and Privacy Feasibility.....14

2.7 Expected Outcomes / Deliverables.....14

CHAPTER 3: REQUIREMENT GATHERING FOR THE PROPOSED SYSTEM.....15

3.1 Introduction to Requirement Gathering.....15

3.2 Functional Requirements.....16

3.3 Non-Functional Requirements.....17

3.4 Hardware, Software, Technologies and Tools Utilized.....18

3.5 Constraints.....19

CHAPTER 4: PROPOSED DESIGN.....20

4.1 Block Diagram of the System.....20

4.2 Modular Design of the System.....22

4.3 Detailed Design....24
4.3.1 Class Design....24
4.3.2 Data Flow....26
4.3.3 Report Structure....27
4.3.4 Statistical Design....27
4.4 Project Scheduling and Tracking....27
4.5 Sustainability-by-Design Decisions....29
CHAPTER 5: IMPLEMENTATION OF THE PROPOSED SYSTEM....30
5.1 Methodology Employed for Development....30
5.2 Algorithms and Flowcharts for Modules Developed....31
5.2.1 Detection and Localization....31
5.2.2 Accuracy Estimation....34
5.2.3 Drift Type Diagnosis....34
5.2.4 Root Cause Analysis....36
5.2.5 Reliability Scoring and Remediation Triage....37
5.3 Dataset Sources and Utilization....39
CHAPTER 6: TESTING OF THE PROPOSED SYSTEM....42
6.1 Introduction to Testing....42
6.2 Types of Tests Considered....42
6.3 Test Case Scenarios....43
6.4 Inference Drawn from the Test Cases....45
CHAPTER 7: RESULTS AND DISCUSSION....46
7.1 User Interface Screenshots for Respective Modules....46
7.2 Performance Evaluation Measures....56
7.3 Input Parameters / Features Considered....57
7.4 Graphical and Statistical Output....58
7.4.1 Drift Type against Baselines....58
7.4.2 Held-out Mechanisms....60
7.4.3 Size of the Delayed Audit....61
7.4.4 Detection and Accuracy Estimation....61
7.4.5 Real Drift....62
7.4.6 Reliability and Triage....65
7.4.7 Computational Cost....66
7.5 Comparison of Results with Existing Systems....66
7.6 Inference Drawn....67

7.7 Sustainability Impact Assessment and Rubric Justification.....67
7.7.1 Scope, Boundary and Evidence.....67
7.7.2 Material Sustainability Topics.....68
7.7.3 Sustainability Targets and Progress.....68
7.7.4 SDG Alignment and Evidence.....69
7.7.5 Sustainability Assessment Rubric.....69
7.7.6 Evidence and Limitations Statement.....70
CHAPTER 8: CONCLUSION.....71
8.1 Limitations.....71
8.2 Future Scope.....72
REFERENCES.....73
APPENDIX – I: PROJECT EVIDENCE AND SUPPORTING DOCUMENTS.....79
A. IPR / Research Publication Evidence.....79
B. Project Competition / Award Certificates.....79
C. Progress Review Sheets.....79
D. Additional Supporting Material.....79
E. Video Presentation.....80

LIST OF FIGURES

Figure..... Page No.

Fig. 1.1: The four drift types on toy data with two classes (circles and triangles). The dashed line is the decision boundary learned on the reference data; in (d) the labels are reversed inside the shaded region.....1

Fig. 3.1: Use cases of ResilientAI for its three groups of direct users.....16

Fig. 4.1: Block diagram of ResilientAI. Stage letters follow the order in which a production window is processed; dashed arrows carry optional or supporting data.....21

Fig. 4.2: Packages of the implementation, arranged in layers; each layer uses the layers below it.....22

Fig. 4.3: Main classes and functions. Hollow triangles mark inheritance, diamonds mark composition and dashed arrows mark use or return values.....25

Fig. 4.4: Flow of data from the public datasets to the dashboard, the paper and this report.....26

Fig. 4.5: Timeline of the project. Bars show the phases, dashed lines the main milestones and the gray band the semester break.....28

Fig. 5.1: Flowchart of detection, accuracy estimation and localization for one window.....32

Fig. 5.2: Decision path that names the drift type. Questions are shown in white boxes and verdicts in gray boxes; the upper row needs no labels and the lower row uses the audit.....34

Fig. 5.3: Class activation maps for the 4 images of CLEAR-10 bucket 10 with the highest out-of-distribution scores. The top row gives each image with its predicted and true class, the bottom row the map of evidence for the predicted class (red is most evidence).....37

Fig. 5.4: Flowcharts of (a) the reliability score of each prediction and (b) the remediation triage.....38

Fig. 7.1: Top of the ACS tab: task and model selectors, the description of the study, the headline numbers and the strip of steps.....46

Fig. 7.2: Step 2a: the label-free accuracy estimate (CBPE) against the measured accuracy, its error per window and the confidence histograms of the reference and of CA 2023.....47

Fig. 7.3: Step 2b: the statistic of every detector per window and, at the bottom, which detector raised an alarm on which window (filled circles).....48

Fig. 7.4: Step 2c: effect size of every feature per window (KS statistic for numeric features, Cramér’s V for nominal ones; a star marks a significant shift) and the share of rows with occupation codes never seen in the reference.....49

Fig. 7.5: Step 3a: change in the mean absolute SHAP value of each feature per window and the reliance on each feature in the reference and in CA 2023.....50

Fig. 7.6: Step 3b: the decision path of the diagnosis for every window, from the label-free tests to the audit, with the verdict on the right; below, the actual, predicted and BBSE-

estimated share of positive cases.....51
Fig. 7.7: Step 4: how well each signal ranks wrong predictions above right ones (area under the ROC curve), the share of predictions per risk label, the error rate inside each label and the weights the fusion learned.....52
Fig. 7.8: Step 5: accuracy recovered by each repair strategy in each window, graded on labeled data (blue means a gain, red a loss, gray not applicable).....53
Fig. 7.9: CLEAR-10 tab, step 3a: class activation maps of the most unusual images of bucket 10.....54
Fig. 7.10: Benchmark tab, step 6: injected against diagnosed drift types, the stage at which each verdict was reached, the effect of the audit size and the correct diagnosis by intensity.....55
Fig. 7.11: Benchmark tab, step 7: share of windows whose drift type is named correctly by our procedure, the rule baselines, the learned meta-classifier and the ablations, with paired tests.....56
Fig. 7.12: Injected (rows) and diagnosed (columns) drift types on the benchmark with LR, in counts of windows; darker cells hold a larger share of their row.....59
Fig. 7.13: Share of concept drift and label noise windows identified correctly at audit sizes of 200, 600 and 1000 labels (ACS Income, design A, LR), with 95% intervals.....61
Fig. 7.14: Measured accuracy of both models and the label-free CBPE estimate on the temporal windows of the four datasets.....63

LIST OF TABLES

Table.....Page No.

Table 1.1: Alignment of the project with the Sustainable Development Goals.....5
Table 2.1: Research papers referred, with the inference drawn for this project.....7
Table 2.2: Patents closest to the project, as checked on 30 October 2026.....10
Table 2.3: Comparison of ResilientAI with existing systems.....11
Table 2.4: Capabilities of the closest methods and of ResilientAI. A tick marks a capability the method provides.....12
Table 2.5: Proposed and actual budget of the project.....13
Table 2.6: Risks and their mitigation.....13
Table 3.1: Functional requirements.....17
Table 3.2: Non-functional requirements, their targets and the values achieved.....18
Table 3.3: Hardware, software, technologies and tools.....18
Table 4.1: Responsibilities, inputs and outputs of the packages.....23
Table 4.2: Milestones, planned dates and dates achieved.....28
Table 5.1: Datasets, drift windows and reference accuracy.....39
Table 5.2: Input features of ACS Income.....40
Table 5.3: Drift mechanisms of the injection benchmark.....41
Table 6.1: Test case scenarios and their results.....43
Table 7.1: Settings of the pipeline and the benchmark.....57
Table 7.2: Share of benchmark windows with the correct drift type (%).....58
Table 7.3: Correct-type rate (Corr., %) and mean accuracy drop (Drop, points) per held-out mechanism of design B.....60
Table 7.4: Alarm rate of each detector by injected type on design A (%), with the 95% interval on the control windows.....61
Table 7.5: Mean absolute error of the four accuracy estimates (percentage points); the best value in each column is in bold.....62
Table 7.6: Detection and diagnosis on the ACS Income windows. The alarms are those of the LR model; Conf. is the confidence test and DC the domain classifier.....63
Table 7.7: Remediation by setting: settings with a significant gain from importance weighting (IW), retraining on the recent window and retraining on fresh labels and the triage’s advice.....65
Table 7.8: Material sustainability topics.....68
Table 7.9: Sustainability targets and progress.....68
Table 7.10: Sustainability assessment rubric of the department.....69

Table 7.11: Self-assessment against the rubric. Weighted points = (rating ÷ 3) × weight.....70

LIST OF ABBREVIATIONS

Abbreviation	Full form
AC	Average confidence
ACS	American Community Survey
ATC	Average thresholded confidence
AUROC	Area under the receiver operating characteristic curve
BBSD	Black-box shift detection
BBSE	Black-box shift estimation
CA, FL, NY, TX, WA	California, Florida, New York, Texas, Washington
CAM	Class activation map
CBPE	Confidence-based performance estimation
CI	Confidence interval
CLEAR	Continual LEArning on Real-World Imagery
CLIP	Contrastive Language-Image Pre-training
CO, PO, PSO	Course outcome, programme outcome, programme specific outcome
CPU, GPU	Central processing unit, graphics processing unit
DDM	Drift detection method
DoC	Difference of confidences
FR, NFR	Functional requirement, non-functional requirement
FWER	Family-wise error rate
GB	Gradient boosting (histogram gradient boosting)
IPR	Intellectual property rights
JSON	JavaScript Object Notation
KS	Kolmogorov–Smirnov
LR	Logistic regression
MMD	Maximum mean discrepancy
OOD	Out of distribution
PSI	Population stability index
RCA	Root cause analysis
ResNet	Residual network
SDG	Sustainable Development Goal
SHAP	Shapley additive explanations
UCI	University of California, Irvine
UI	User interface

ABSTRACT

Machine learning models are trained once but used for months or years. In that time the data they see in production slowly stops looking like the data they were trained on. When this happens accuracy falls, often without anyone noticing, because the true labels that would reveal the loss arrive weeks or months later. Existing monitoring tools can raise an alarm when the data change, but they rarely say what kind of change took place. This matters because each kind calls for a different response: a change in the inputs alone may need only reweighting, a change in the class balance needs recalibration, a change in the relationship between inputs and labels needs fresh labels and corrupted labels point to a fault in the labeling process.

ResilientAI is an end-to-end monitoring system that answers these questions for every new batch of production data. It tests whether the data have changed, estimates the lost accuracy from the model's own confidence, finds the features that moved and names the drift type with a sequence of statistical tests that share one stated error rate. A first stage needs no labels and separates input shift from class balance shift. A second stage, new in this project, uses a few hundred late labels to separate concept drift from label noise. The system also explains the change, scores how far each single prediction can be trusted and advises whether a cheap retrain would help. An interactive dashboard presents every result.

On a benchmark of drift injected into two census tasks, the system named the drift type in 85.7 to 94.3% of windows and stayed at 62.4 to 81.5% on drift mechanisms it was not designed for, where a learned rule fell to 31.1 to 51.2%. On four real datasets with documented changes it raised no false alarm on any control window and traced the known events.

Keywords: concept drift, dataset shift, model monitoring, drift diagnosis, hypothesis testing, machine learning operations

CHAPTER 1: INTRODUCTION

This chapter introduces the problem that ResilientAI addresses: deployed machine learning models that lose accuracy silently when their data change. It explains why the problem matters, defines it precisely, reviews the systems that exist today and their gaps and ends with the relevance of the project and its link to the United Nations Sustainable Development Goals.

1.1 Introduction

A machine learning model is built from a fixed set of historical examples and then put to work on new data, often for a long time. A credit model, a demand forecast or a medical triage tool is trained once, checked on held-out data and deployed. The world it was trained on does not stand still, however. Customers change, sensors age, survey questions are rewritten and occupations are reclassified. The data that reach the model in production drift away from the data it learned from and its predictions get worse. This is known as dataset shift or data drift [1], [2].

The difficulty is that the loss is silent. A deployed classifier keeps producing confident predictions after its accuracy has fallen and the true labels that would expose the loss tend to arrive weeks or months later, if at all [3]. Monitoring therefore has to work from what is available at prediction time: the inputs, the model's outputs and its confidence. Model monitoring has become a recognised part of machine learning operations for this reason [4]–[6].

Not every drift is the same. The right response depends on its kind. Moreno-Torres et al. [1] distinguish three types of dataset shift and this project adds a fourth. Fig. 1.1 shows the four on a small two-dimensional example.

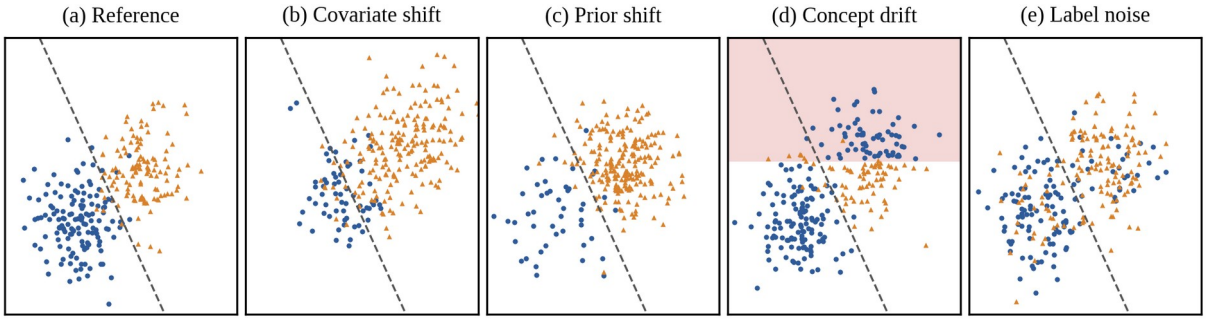


Fig. 1.1: The four drift types on toy data with two classes (circles and triangles). The dashed line is the decision boundary learned on the reference data; in (d) the labels are reversed inside the shaded region

Panel (a) is the reference data the model was trained on. Under covariate shift (b) the inputs move but the rule that maps inputs to labels holds, so reweighting the training data is often enough to adapt the model. Under prior shift (c) only the class balance changes, which recalibration usually fixes. Concept drift (d) alters the mapping itself: the same inputs now carry different labels in some region and nothing short of fresh labels will repair the model. Label noise (e) is different again. The true relationship is unchanged, but the labels used to grade the model are corrupted, so the fault lies in how labels are collected rather than in the model.

ResilientAI is a monitoring system built around this idea. It takes a deployed model, its labeled training data and a stream of unlabeled production data and for every production window it answers the questions a monitoring team asks in turn. Has the data changed? How much accuracy has been lost? Which features moved? What type of drift is it and what probably caused it? Which predictions should not be trusted? Would a cheap retrain recover the loss? The implementation, its dashboard and the research paper written from this project call the pipeline Data Drift Autopsy; ResilientAI is the name of the project as a whole.

The rest of this report follows the template of the department. Chapter 2 surveys the literature and patents, Chapter 3 gathers the requirements and Chapter 4 presents the design. Chapters 5 and 6 cover the implementation and its testing, Chapter 7 presents the results and Chapter 8 concludes.

1.2 Motivation

The motivation for this project came from a simple observation made while working with census data. A model that predicts whether a person earns more than $50,000, trained on 2014 data from California, loses accuracy year after year when it is applied to later years. The monitoring tools we tried raised alarms on almost every later year, which was correct, yet none of them could say whether the cause was a change in the population, a change in the survey or a change in what the label means. A team receiving such an alarm still has to work out by hand what to do.

There are three practical reasons to want more than an alarm. First, labels are expensive: a team should not have to label every production row to find out whether its model still works. Second, the wrong response wastes effort. Retraining on fresh labels does not help when the labels themselves are noisy and reweighting does not help when the concept has changed. Third, alarms without a stated error rate cannot be trusted. A monitor that tests dozens of features at a fixed threshold will raise false alarms by chance and teams quickly learn to ignore it.

There was also a research motivation. The literature offers good tools for detecting a shift and for estimating accuracy without labels, but, as Chapter 2 shows, no published method separates concept drift from label noise with a small label budget and type diagnoses are rarely tested on drift they were not designed for. Closing that gap with tools a monitoring team can actually run seemed both useful and achievable within a capstone project.

1.3 Problem Definition

The project addresses the following problem. Given a deployed classifier, the labeled reference data it was built from and a sequence of unlabeled production windows, the system must decide for each window whether the data have drifted. When they have, it must estimate the accuracy the model is likely to have, locate the features that moved and name the drift type as one of covariate shift, prior shift, concept drift or label noise, so that the matching response can be chosen. Production labels may be used only in a small optional audit of a few hundred rows that arrive late. Every decision must come with a stated error rate, so that a window without drift receives a false verdict with a probability fixed in advance.

The target users are machine learning engineers and monitoring teams who own deployed models and the teams that label data for them. The scope covers tabular classifiers and image classifiers whose images are first mapped to embeddings by a pretrained network. Text models, deep models trained end to end on the production stream and real-time streaming at sub-second latency are outside the scope.

The intended outcome is a working system, evaluated on data where the true drift type is known and on real data whose changes are documented, together with a dashboard that presents the findings to a person who must act on them.

1.4 Existing Systems

Several kinds of system already watch deployed models. They differ in what they look at and in what they report.

- **Error-rate monitors** such as the drift detection method (DDM) [7] and ADWIN [8] track the error rate of the model and raise an alarm when it rises. They need the true label of every prediction soon after it is made.
- **Label-free shift detectors** compare the production inputs or the model’s outputs with the reference data through two-sample tests: univariate tests, kernel tests such as the maximum

mean discrepancy [9], classifier tests [10] and black-box shift detection on the model's outputs [3].

- **Label shift estimators** such as black-box shift estimation [11] estimate how the class balance has changed, assuming that each class keeps its own input distribution.
- **Performance estimators** such as confidence-based performance estimation [12], [13] estimate the accuracy of the model from its confidence.
- **Monitoring libraries and platforms** package such tests with dashboards, explanations and alerts [14]. Several companies have also patented drift monitoring methods, which Chapter 2 reviews.

1.5 Lacunae in Existing Systems

The existing systems leave several gaps, each supported by the survey in Chapter 2.

1. **They detect but do not diagnose.** Label-free detectors report that a shift happened, not what type it was [3]. Error-rate monitors notice concept drift but cannot tell it apart from label noise, because both raise the error rate [7].
2. **Diagnosis needs labeled production data.** The most complete type diagnosis, DetectShift [15], tests each part of the joint distribution of inputs and labels but needs labels for the new data, which its authors acknowledge as a limitation.
3. **Label noise is not treated as a drift type.** None of the methods we found separates concept drift from label noise using a small audit of late labels.
4. **Error rates are not controlled across decisions.** Monitors that test many features and several detectors at a fixed threshold do not control the chance of at least one false alarm across all of them.
5. **Methods are graded on their own benchmarks.** Injected drift gives a ground truth but may favor the method being tested. Type diagnoses are rarely checked on drift mechanisms held out from their design.
6. **Accuracy estimates assume that confidence stays honest.** Confidence-based estimates assume that the link between confidence and accuracy survives the shift, which often fails [16].

1.6 Relevance of the Project

The project is relevant wherever a model keeps running after its training data have aged. Banks, insurers, public agencies, hospitals and manufacturers all deploy classifiers on populations and processes that change. For them, knowing the type of a drift turns an alarm into an action: reweight, recalibrate, collect fresh labels or check the labeling process. Because the system needs labels only for a small audit, it also lowers the cost of monitoring for teams that cannot label their production data.

The beneficiaries are, first, machine learning engineers and monitoring teams, who get a ranked and explained diagnosis instead of a bare alarm. The people affected by model decisions benefit indirectly when a failing model is caught and repaired sooner. The project also adds to research on dataset shift: its audit test, its error-controlled procedure and its held-out benchmark are described in a research paper prepared for publication (Appendix I).

1.7 Alignment with the United Nations Sustainable Development Goals (SDGs)

The project is linked to two of the seventeen Sustainable Development Goals of the United Nations: Goal 9 (Industry, Innovation and Infrastructure) and Goal 12 (Responsible Consumption and Production) [17]. Table 1.1 ties each goal to a specific target, to the part of the project that serves it and to the evidence in this report. The last column states whether each link is a design intent, a tested output or an expected contribution, as the template asks.

Table 1.1: Alignment of the project with the Sustainable Development Goals

SDG and name	Target	Project component	Contribution	Indicator and evidence
SDG 9: Industry, Innovation and Infrastructure	9.5: enhance scientific research and upgrade technological capabilities	Audit test for concept drift against label noise; error-controlled diagnosis; held-out benchmark	New method and open evaluation design for a monitoring problem without a published solution (tested output)	Correct-type rates in Section 7.4; research paper in Appendix I
SDG 9: Industry, Innovation and Infrastructure	9.4: upgrade infrastructure for greater reliability and efficiency	End-to-end monitoring of deployed models with a stated false alarm rate	Silent model failure is caught and explained instead of going unnoticed (expected contribution)	No false alarm on any real control window; false alarm rate on benchmark controls in Section 7.4
SDG 12: Responsible Consumption and Production	12.2: efficient use of natural resources	Remediation triage, label-free decisions, frozen image embeddings, capped parallel workers	Avoids needless full retraining and most labeling; reuses computation (design intent, partly tested)	Triage accuracy in Section 7.4; run time and energy estimates in Section 7.7

SDG and name	Target	Project component	Contribution	Indicator and evidence
SDG 12: Responsible Consumption and Production	12.6: integrate sustainability information into reporting	Sustainability assessment of the project with indicators and stated limits	Reports energy, emissions, privacy and inclusion with method and evidence (tested output)	Section 7.7

The link to Goal 9 is the most direct. The project develops and evaluates a new technical capability for keeping deployed models reliable, which is what target 9.5 describes and its monitoring makes digital infrastructure that depends on models more dependable, in the spirit of target 9.4. The link to Goal 12 rests on efficiency: a monitor that tells a team when retraining will not help and that needs labels for only a few hundred rows, saves computation and human labeling effort. The size of these savings in a real deployment has not been measured, so this report claims them as design intent and expected contribution only. Section 7.7 returns to both goals with the evidence gathered during the project.

CHAPTER 2: LITERATURE SURVEY

Before designing the system we studied how others have approached drift: how the types of drift are defined, how drift is detected, how a change in the class balance is estimated, how accuracy can be estimated without labels and how the findings are explained. This chapter reports that survey, a search of granted and published patents, the conclusions we drew, a comparison of ResilientAI with the closest existing systems and an analysis of whether the project was feasible.

2.1 Brief Overview of the Literature Survey

The survey had one guiding question: which parts of drift monitoring are solved and which are not? We searched IEEE Xplore, the ACM Digital Library, SpringerLink, the proceedings of NeurIPS, ICML and ICLR, arXiv and Google Scholar with the keywords dataset shift, covariate shift, label shift, prior shift, concept drift, label noise, drift detection, two-sample test, performance estimation without labels, model monitoring and machine learning operations. Two surveys of concept drift [18], [19], a formal characterization of the kinds of drift [20] and a book on dataset shift [2] served as starting points and we followed their references forward and backward.

A paper was kept if it proposed or evaluated a method for detecting, estimating or explaining a change between training and production data, or if it supplied data with a documented shift. We preferred peer-reviewed work and recent papers, but kept older ones that remain the standard reference for a method. Documentation of monitoring software was read for context. Table 2.1 summarizes the papers that shaped the design.

2.2 Related Works / Research Papers Referred

The papers fall into five groups. Each group below is introduced briefly and Table 2.1 lists the method, findings and limitations of each paper together with the inference we drew for this project.

Table 2.1: Research papers referred, with the inference drawn for this project

Paper	Method / approach	Key findings	Limitations / inference
Moreno-Torres et al., 2012 [1]	Taxonomy of dataset shift by the part of the joint distribution that changes	Defines covariate shift, prior shift and concept shift precisely	Gives definitions, not tests; we test each definition directly
Gama et al., 2004 [7]	Drift detection method: tracks the error rate of the model online	Detects concept drift as a rise in errors	Needs a label for every prediction and cannot tell concept drift from label noise
Bifet and Gavaldà, 2007 [8]	ADWIN: adaptive windows over a data stream	Detects change in a stream with bounds on false alarms	Watches one statistic; says that a change happened, not its type

Paper	Method / approach	Key findings	Limitations / inference
Gretton et al., 2012 [9]	Kernel two-sample test (maximum mean discrepancy)	Detects differences between multivariate distributions	Sees joint changes that per-feature tests miss; used as one block of our label-free stage
Lopez-Paz and Oquab, 2017 [10]	Classifier two-sample test	A classifier that separates two samples better than chance shows that they differ	Used as a baseline detector; gives no drift type
Rabanser et al., 2019 [3]	Compares univariate, kernel, classifier and black-box shift detectors on injected shift	Black-box shift detection on the model outputs works best overall	Detects shift but does not name its type; their injected benchmark inspired ours
Lipton et al., 2018 [11]	Black-box shift estimation of the new class prior from the confusion matrix	Recovers label shift from any classifier	Assumes the inputs within each class do not change; we test that assumption
Maia Polo et al., 2023 [15]	DetectShift: tests each part of the joint distribution	Names shift types with error control	Needs labeled production data; does not treat label noise
Roschewitz et al., 2025 [21]	Resamples reference data to the estimated prevalence and re-tests the features	Separates prevalence shift from covariate shift on medical images without labels	Our label-free stage follows the same idea with weights and one error-controlled family
Hendrycks and Gimpel, 2017 [22]	Maximum softmax probability as a baseline signal	Confidence separates correct from wrong predictions and in- from out-of-distribution inputs	Basis of average confidence, an accuracy estimate we compare
Guillory et al., 2021 [23]	Difference of confidences between reference and target	Predicts accuracy change under shift	Assumes confidence stays calibrated; compared as an estimator
Garg et al., 2022 [24]	Average thresholded confidence	Accurate accuracy estimates on many shifts	Fails when the confidence-accuracy link breaks; compared as an estimator
Kivimäki et al., 2026 [12]	Confidence-based performance estimation with calibrated confidence	Estimates the confusion matrix and accuracy without labels	Our main accuracy estimate, read as an upper bound when confidence misleads
Ovadia et al., 2019 [16]	Evaluates predictive uncertainty under dataset shift	Models stay confident while accuracy falls	Explains why label-free estimates can be optimistic
Lundberg and Lee, 2017 [25]	Shapley additive explanations	Attributes each prediction to its input features	Used to compare what the model relies on before and after a shift
Ding et al., 2021 [26]	Folktables: prediction tasks from the American Community Survey	Real distribution shift across years and states	Our main source of real tabular drift
Lin et al., 2021 [27]	CLEAR: a time-ordered image benchmark from Flickr	Natural drift of image content over a decade	Our source of real image drift

Drift types. The definitions of Moreno-Torres et al. [1] describe each type by the part of the joint distribution of inputs and labels that changes and Webb et al. [20] extend them to describe how drift unfolds in time. DetectShift [15] is the only method we found that tests each part with error control, but it needs labels for the new data. These papers gave the project its vocabulary and its main requirement: a type diagnosis that works from unlabeled data plus a small audit.

Drift detection. Detectors fall into two families. Error-rate detectors such as DDM [7] and ADWIN [8] need labels, while label-free detectors compare inputs or outputs with two-sample tests [9], [10]. Rabanser et al. [3] compared the label-free tests systematically and found black-box shift

detection on the model outputs best overall. Every one of these tests reports that a shift happened, not its type, so we use them as building blocks.

Label shift. Black-box shift estimation [11] recovers the new class prior under the assumption that each class keeps its own input distribution. Roschewitz et al. [21] turned that assumption into a test, which is the idea behind our label shift consistency test.

Performance estimation without labels. Average confidence [22], difference of confidences [23], average thresholded confidence [24] and confidence-based performance estimation [12] all estimate accuracy from what the model outputs. Ovadia et al. [16] showed that models often stay confident while their accuracy falls, so the estimates can be too optimistic exactly when they matter most.

Explanation and data. Shapley values [25] and class activation maps [28] explain what a model relies on and out-of-distribution scores such as the Isolation Forest [29] flag unusual inputs. Folktables [26] and CLEAR [27] provide real shifts whose causes can be traced, although nobody knows their exact type. Benchmarks of real distribution shift [30]–[32] show how often models fail under such shifts.

2.3 Patent Search

To check whether the idea of the project is already protected, we searched Google Patents and Espacenet on 30 October 2026 with the search terms “data drift” and “concept drift” combined with “machine learning model”, “monitoring”, “covariate shift”, “label shift” and “root cause”. We read the claims and descriptions of the most relevant results and recorded their legal status as shown by Google Patents, which presents status as an assumption rather than a legal conclusion. Table 2.2 lists the six patents closest to this project: four from the United States and two from the European Patent Office.

Table 2.2: Patents closest to the project, as checked on 30 October 2026

Number and office	Title	Assignee	Status and date	What it claims and how it differs
US 10 762 444 B2 [33]	Real-time drift detection in machine learning systems and applications	Quickpath, Inc.	Granted, 1 Sep. 2020; active	Compares live score distributions with training scores and alerts on significant drift; does not name a drift type
US 11 568 167 B2 [34]	Detecting drift between data used to train and data used to execute a machine learning model	Arthur AI, Inc.	Granted, 31 Jan. 2023; active	Scores each production row against a density model of the training data and triggers retraining past a threshold; no type diagnosis or error control
US 12 199 847 B2 [35]	Anomaly detection of model performance in an MLOps platform	Cisco Technology, Inc.	Granted, 14 Jan. 2025; active	Correlates drops in measured accuracy with feature shifts and tags the most correlated feature as the probable cause; needs accuracy measurements
US 12 282 384 B2 [36]	Systems and methods for detecting drift	Jio Platforms Limited	Granted, 22 Apr. 2025; active	Compares model predictions with reference values and flags drift when the error passes a threshold; does not separate drift types
EP 4 459 516 A1 [37]	Drift detection for machine learning operations	Siemens AG	Application published 6 Nov. 2024; withdrawn	Compares normal and outlier subsets of old and new measurements to predict drift and choose retraining or remodeling
EP 4 577 950 A1 [38]	Real time detection, prediction and remediation of machine learning model drift in asset hierarchy	Hitachi Vantara LLC	Application published 2 Jul. 2025; pending	Detects data and concept drift in sensor time series with the population stability index and clustering, forecasts drift and remediates

The patents show that drift detection and alerting are well protected and two of them go as far as suggesting a likely cause. None of them names the four drift types of this project, none separates concept drift from label noise and none controls the error rate across its decisions. The audit test and the error-controlled diagnosis of ResilientAI therefore do not appear to fall within the claims we read. This is a technical reading for the purpose of this report, not a legal opinion on freedom to operate.

2.4 Inference Drawn

Three conclusions came out of the survey and shaped the design of ResilientAI.

First, detection is a solved part of the problem and diagnosis is not. Good label-free tests exist and the system should reuse them rather than invent new detectors. The new work should go into naming the type of a detected shift.

Second, the gap that matters most is between concept drift and label noise. Both leave the inputs untouched, so no label-free test can see them and both raise the error rate, so an error-rate monitor

confuses them. Some labels are therefore unavoidable. The question the project had to answer was how few labels would do and how to use them so that the two types come apart. The answer we developed, described in Chapter 5, is to ask whether the change in errors depends on an input feature beyond what the model's own confidence explains.

Third, a diagnosis must be judged fairly. A method tuned on its own benchmark can look good by construction. This led to a benchmark with two designs: one whose drift partly matches how the tests work and one with mechanisms held out from the design of the procedure. It also led to the decision to control the error rate of every decision, so that the false alarm rate is known before any data are seen.

2.5 Comparison with Existing Systems

Table 2.3 compares ResilientAI with the existing systems closest to it and Table 2.4 summarizes their capabilities side by side.

Table 2.3: Comparison of ResilientAI with existing systems

System / paper	Features / method	Strengths	Limitations	Difference in proposed work
Error-rate monitors (DDM, ADWIN) [7], [8]	Track the error rate online	Simple; detect concept drift	Need all labels; cannot separate concept drift from label noise	Labels only for a small audit; separates the two
Black-box shift detection [3]	Two-sample tests on model outputs	Strong, label-free detection	No drift type, no accuracy estimate	Detection is one stage of a full diagnosis
Black-box shift estimation [11]	Estimates the new class prior	Label-free; works with any classifier	Assumes pure label shift	Tests the assumption before naming prior shift
CBPE and monitoring libraries [12], [14]	Accuracy estimates, detectors, dashboards	Practical, packaged tools	No type diagnosis; estimates trust confidence	Adds diagnosis; reads the estimate as an upper bound
DetectShift [15]	Tests each part of the joint distribution	Error-controlled type tests	Needs labeled production data; no label noise	Unlabeled data plus a few hundred late labels; adds label noise
Roschewitz et al. [21]	Prevalence-adjusted re-testing of features	Separates prior from covariate shift without labels	Two types only	Same idea inside a four-type, error-controlled procedure
Drift patents [33], [35], [36]	Distribution comparison, thresholds, correlation with accuracy	Deployed in industry	Thresholds without error control; no type diagnosis	Hypothesis tests with a stated error rate; four types

Table 2.4: Capabilities of the closest methods and of ResilientAI. A tick marks a capability the method provides

Method	Error rate stated	Accuracy estimate	Localizes features	Types named	Concept vs noise	Held-out drift
BBSD [3]	√	–	limited	–	–	–
BBSE [11]	√	–	–	prior	–	–
CBPE [12]	–	√	–	–	–	–
DetectShift [15]	√	–	–	needs labels	–	–
Roschewitz et al. [21]	√	–	–	prior, covariate	–	–
ResilientAI	√	√	√	four	√	√

Each existing method covers one or two of the columns. Roschewitz et al. come closest, separating prior from covariate shift without labels, while DetectShift names types only when the production data are labeled. ResilientAI is the only one that covers every column and in particular the only one that separates concept drift from label noise and is tested on held-out drift.

2.6 Feasibility Analysis

Before committing to the project we checked whether it could be built and run with the resources available to a student team and whether it raised any legal or ethical concern. The analysis below covers each dimension the department asks for.

2.6.1 Technical Feasibility

Every building block of the system is a well-understood statistical test or machine learning model with a mature open-source implementation: scikit-learn [39] for the models, SciPy and NumPy [40], [41] for the tests, SHAP [25] for explanations and PyTorch for the image network. The data are public. The heaviest computation, the injection benchmark, took 1.6 hours on a desktop processor with 20 cores and no production window took longer than 90 seconds to analyze (Section 7.4.7). The project was therefore technically feasible on the team’s own hardware.

2.6.2 Economic and Financial Feasibility

The project needed no purchase. Table 2.5 gives the budget, with the electricity cost estimated from the measured running time and the processor’s rated maximum power.

Table 2.5: Proposed and actual budget of the project

Item	Basis	Cost (₹)
Development workstation (Intel Core i5-14600K, 32 GB memory, NVIDIA RTX 4070)	Already owned by a team member; no purchase	0
Software: Python, scikit-learn, SciPy, PyTorch, SHAP, Streamlit, LibreOffice	Open-source licences	0
Datasets: Folktables ACS, UCI gas sensor array, CLEAR-10	Free for research use	0
Cloud computing and paid services	Not used	0
Electricity for the experiments	0.43 kWh at an assumed ₹10 per kWh	4
Total		4

2.6.3 Cost-effectiveness Analysis

For a team that deploys models, the alternative to a monitor like ResilientAI is to label production data regularly and measure accuracy directly. Labeling every window of several thousand rows is far more expensive than labeling an audit of 600 rows and Section 7.4 shows that more audit labels add little beyond that number. The triage further avoids the cost of retraining when retraining would not help. The monitor itself runs on ordinary hardware within seconds to minutes per window, so its running cost is small next to the cost of either labeling or a model failure that goes unnoticed.

2.6.4 Operational Feasibility and Risk Analysis

The system runs as a set of Python scripts that write one report per study and a dashboard that reads those reports, so it fits into an existing batch workflow without new infrastructure. Table 2.6 lists the main risks we identified and how each was mitigated.

Table 2.6: Risks and their mitigation

Risk	Likelihood	Mitigation
False alarms from testing many features and detectors	High	Weighted Holm procedure keeps the family-wise error rate at α; checked by simulation and on control windows
A diagnosis tuned to its own benchmark	Medium	Held-out design B; diagnosis frozen before design B was first run
Errors in the statistical code	Medium	113 automated tests; code review; two errors found and every result regenerated
Label-free accuracy estimate misleads under shift	Medium	Reported as an upper bound; four estimators compared
Long runs overload the machine	Medium	Capped worker counts and one heavy job at a time
Data access or format changes (2020 ACS release, renamed variables)	Low	Cached data; documented mappings (RELSHIPP to RELP)

2.6.5 Environmental and Sustainability Feasibility

The project uses no special hardware and no cloud resources. The experiments consumed about 0.43 kWh of electricity in total, an upper-bound estimate explained in Section 7.7, which corresponds to about 0.31 kg of carbon dioxide at the average emission factor of the Indian grid. Design choices that keep computation low are described in Section 4.5.

2.6.6 Legal, Ethical and Privacy Feasibility

All datasets are public and licensed for research. The American Community Survey microdata are released by the United States Census Bureau under its terms of use and contain no names or addresses; the Folktables code that downloads them is under the MIT licence. The UCI gas sensor dataset and the CLEAR benchmark are released under the Creative Commons Attribution 4.0 licence. No personal data were collected from anyone, no human subjects took part and everything runs locally, so no data leave the machine. The census tasks involve sensitive attributes such as sex and race, which appear as model inputs only because they are part of the standard Folktables tasks; the project does not make decisions about individuals.

2.7 Expected Outcomes / Deliverables

The project set out to deliver the following.

1. A Python package that implements detection, accuracy estimation, localization, drift type diagnosis, root cause analysis, reliability scoring and remediation triage.
2. An interactive dashboard that walks through these steps for real and injected drift.
3. An injection benchmark with known drift types, including drift mechanisms held out from the design.
4. An evaluation on four real datasets with documented changes, with every reported number generated from the study outputs.
5. A research paper describing the method and its evaluation.
6. This report.

All six were delivered. Chapter 7 presents the evidence and Appendix I records the status of the research paper.

CHAPTER 3: REQUIREMENT GATHERING FOR THE PROPOSED SYSTEM

A monitoring system is useful only if it answers the questions its users actually ask and does so within limits they can accept. This chapter describes how the requirements of ResilientAI were gathered, lists the functional and non-functional requirements that the rest of the report traces and records the hardware, software and constraints of the project.

3.1 Introduction to Requirement Gathering

The requirements came from four sources. The literature survey of Chapter 2 showed what existing tools already do and where they stop. The documentation of monitoring libraries and the patents of Section 2.3 showed what industry monitors report. The mentor reviews of the project, held at each stage, turned a broad idea into testable targets and pushed us to state an error rate for every decision. Finally, building early prototypes and showing them as a dashboard exposed what a person needs to see before acting on an alarm.

Three groups of stakeholders shape the requirements. Machine learning engineers register a model with its reference data, run the monitor and export its reports. Monitoring teams read the results: they want to know whether the model is still accurate, what changed and what to do about it. Labeling teams supply the small audit of late labels when it is requested. The people affected by the model’s decisions are indirect stakeholders whose interest is that a failing model is caught early. Fig. 3.1 shows the main use cases of the three direct groups.

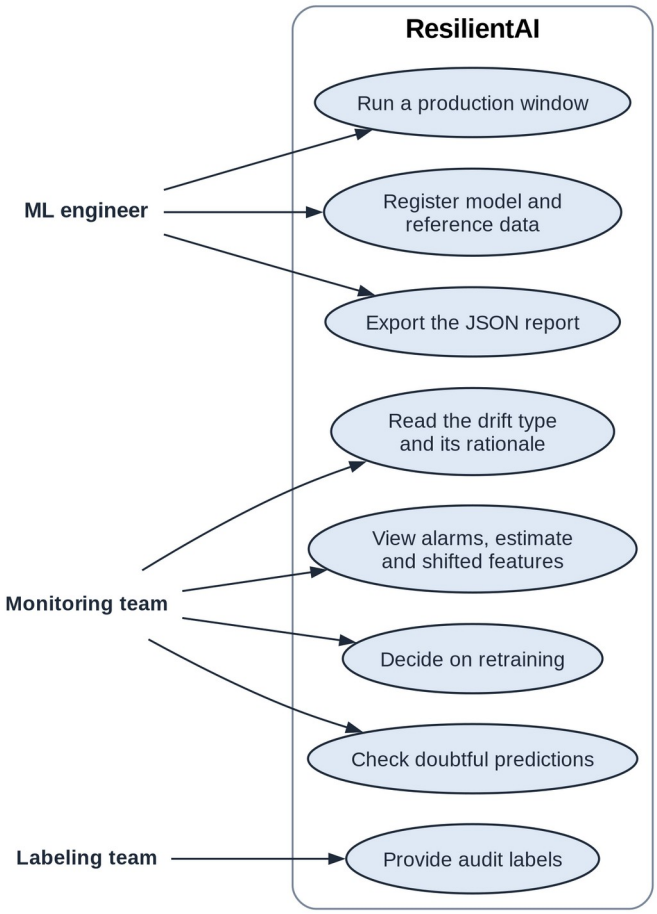


Fig. 3.1: Use cases of ResilientAI for its three groups of direct users

The machine learning engineer sets the system up and runs it. The monitoring team reads alarms, accuracy estimates and shifted features, reads the drift type with the reason the system gives for it, checks doubtful predictions and decides on retraining. The labeling team only provides audit labels. The scope of the requirements follows from the problem definition of Section 1.3: tabular and image classifiers, batch windows of production data and a label budget of a few hundred rows per window.

3.2 Functional Requirements

Table 3.1 lists the functional requirements. Each has an identifier that the test cases of Chapter 6 refer to, a priority and the module of the implementation that meets it.

Table 3.1: Functional requirements

ID	Requirement	Priority	Module
FR-01	Load a deployed model, labeled reference data and unlabeled production windows; validate their schema and record which features are nominal.	High	data, core
FR-02	Split the reference data into disjoint training, calibration and test parts, stratified by class.	High	experiments
FR-03	Map images to embeddings with a frozen pretrained network so that image data can be tested like tabular data.	Medium	extractors, data
FR-04	Test each window for a change in the inputs and in the model outputs (Kolmogorov–Smirnov, population stability index, maximum mean discrepancy, confidence test) with a stated significance level.	High	detectors
FR-05	Estimate the accuracy of the model on each window without labels (confidence-based performance estimation; average confidence, difference of confidences and thresholded confidence for comparison).	High	estimation, detectors
FR-06	Locate the shifted features with a per-feature test and an effect size and report categories never seen in the reference.	High	localizers
FR-07	Name the drift type as no drift, covariate shift, prior shift, concept drift or label noise, with the stage and the reason for the verdict, holding the family-wise error rate at the chosen level.	High	diagnosis
FR-08	Use an optional audit of late labels to test for an accuracy drop and for errors that depend on a feature beyond the model’s confidence.	High	diagnosis
FR-09	Explain a shift by the change in what the model relies on: Shapley values for tabular models and class activation maps for image models.	Medium	rca
FR-10	Score the risk of error of every single prediction from its confidence, its instability under small perturbations and an out-of-distribution score.	Medium	reliability
FR-11	Advise whether a cheap retrain would recover accuracy, using importance weighting and, where available, an earlier labeled window.	Medium	remediation
FR-12	Write every result of a study to one machine-readable report.	High	experiments
FR-13	Present the results step by step in an interactive dashboard for real and injected drift.	High	dashboard
FR-14	Inject drift of a known type and intensity into real data and grade the diagnosis against the truth.	High	benchmark

3.3 Non-Functional Requirements

Table 3.2 gives the non-functional requirements as measurable targets, together with the way each was checked and the value achieved.

Table 3.2: Non-functional requirements, their targets and the values achieved

ID	Quality	Target	How checked	Achieved
NFR-01	Reliability of alarms	Each detector's false alarm rate on windows without drift at or below $\alpha = 0.05$	95% CI over 200 benchmark control windows	Every CI contains α or lies below it
NFR-02	Reliability of the diagnosis	Probability of a drift verdict on a window without drift at most $2\alpha = 10\%$	Benchmark control windows	6.5%
NFR-03	Error control	Family-wise error of the label-free decisions at most α	Simulation with 200,000 replicates	4.9%
NFR-04	Performance	Analysis of one window well within a daily monitoring cycle	Measured run time per window	At most 90 s
NFR-05	Label economy	Labels needed only for the optional audit	Design; audit-size study	600 labels per window
NFR-06	Reproducibility	Every result repeatable exactly	Seeded randomness; all reported numbers generated from the study outputs	Met
NFR-07	Correctness of the code	All automated tests pass	pytest	113 of 113 passed
NFR-08	Usability	Each dashboard step answers one stated question with a plain-language summary	Review of the dashboard	Met (Section 7.1)
NFR-09	Maintainability	Modules separated by responsibility, typed configuration, documented data contracts	Code structure (Section 4.2)	Met
NFR-10	Privacy and security	No personal data collected; all processing local	Design; datasets are public	Met
NFR-11	Resource use	No overload of the workstation during long runs	Capped workers; one heavy job at a time	Met

3.4 Hardware, Software, Technologies and Tools Utilized

Table 3.3 lists the hardware and software used to build and run the system, with the versions installed when the results in this report were produced.

Table 3.3: Hardware, software, technologies and tools

Category	Tool / specification	Purpose / version
Hardware	Intel Core i5-14600K (20 threads), 32 GB memory, NVIDIA GeForce RTX 4070 (12 GB)	Development, experiments, image embeddings on the graphics card
Software / OS	Linux (Arch-based distribution, kernel 7.2); Python 3.14	Operating system and language
Languages / frameworks	NumPy 2.4, pandas 2.3, SciPy 1.17, scikit-learn 1.8	Data handling, statistical tests, models
Languages / frameworks	PyTorch 2.11, torchvision 0.26, OpenCLIP 3.3	Image network (ResNet-18), CLIP concept probe
Languages / frameworks	SHAP 0.50	Shapley value explanations

Category	Tool / specification	Purpose / version
Database / cloud / APIs	JSON files, one per study; Folktables 0.0.12 for the census data; no cloud services	Storage of results and data access
Visualization / UI	Streamlit 1.54, Plotly 6.5, Matplotlib 3.10, Graphviz	Dashboard and figures
Testing / analytics / version control	pytest 9.0, Git with two repositories (code and paper)	Automated tests and version control
Documentation	LibreOffice 26.8, Markdown sources	This report and the paper versions

3.5 Constraints

The project worked within the following constraints and assumptions.

- **Labels.** Production labels are assumed to arrive late and to be expensive, so they may be used only in a small audit. The audit is assumed to be a random sample of the window.
- **Data.** Only public datasets could be used. Their drift types are not known exactly, which is why an injection benchmark was needed. The 2020 release of the American Community Survey has no standard version, so 2020 is missing from the timelines.
- **Models.** The system watches the model from outside: it needs predictions and class probabilities, not the internals of the model, except for the explanations.
- **Compute and time.** All work had to run on one workstation within the semester. Long benchmark runs were scheduled one at a time with capped parallel workers.
- **Batch setting.** Windows are batches of hundreds to thousands of rows. The tests are not designed for streams that need a verdict every few seconds.
- **Ethics and privacy.** No personal data could be collected and the sensitive attributes in the census tasks are used only as model inputs of standard public tasks.

CHAPTER 4: PROPOSED DESIGN

This chapter turns the requirements of Chapter 3 into a design. It starts from a block diagram of the whole system, breaks the system into modules, shows the main classes and the flow of data between them, presents the schedule the project followed and ends with the design decisions taken to keep the system economical in computation and labels.

4.1 Block Diagram of the System

The system is organized as a pipeline of stages, each answering one of the questions a monitoring team asks. Fig. 4.1 shows the stages and the data that pass between them.

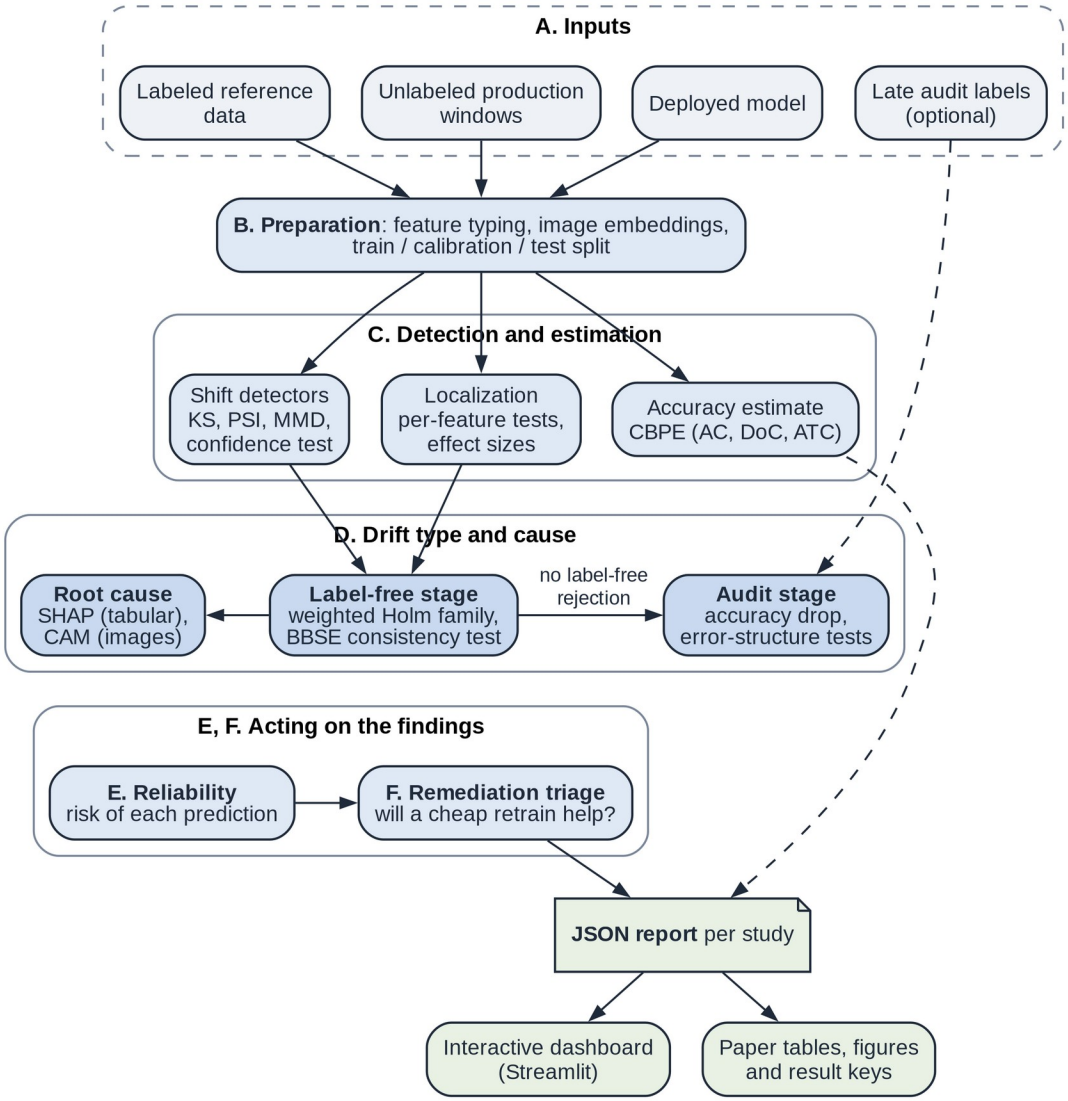


Fig. 4.1: Block diagram of ResilientAI. Stage letters follow the order in which a production window is processed; dashed arrows carry optional or supporting data

A. Inputs. The system takes the deployed model, the labeled reference data it was built from, the unlabeled production windows and, optionally, a few hundred late labels for each window.

B. Preparation. The data are brought into a common form. Feature types are recorded, so that nominal codes such as an occupation code are never treated as numbers. Images are mapped to embeddings by a frozen pretrained network. The reference data are split into training, calibration and test parts.

C. Detection and estimation. Shift detectors compare each window with the calibration part, the accuracy is estimated from the model's confidence and the shifted features are located with per-feature tests.

D. Drift type and cause. The label-free stage of the diagnosis decides whether anything moved and whether a change in the class balance alone explains it. When no label-free test rejects and an audit is available, the audit stage decides between concept drift, label noise and no drift. Root cause analysis compares what the model relies on before and after the shift.

E and F. Acting on the findings. The reliability stage scores the risk of each single prediction and the remediation triage advises whether a cheap retrain would recover the accuracy.

Every result of a study is written to one report in JavaScript Object Notation (JSON). The dashboard reads these reports and so does the build that generates the tables, figures and numbers of the paper and of this report.

4.2 Modular Design of the System

The implementation is a Python package of 57 modules and 11,057 lines in 13 subpackages, plus the dashboard and the build scripts. Fig. 4.2 shows the packages as layers, where each layer uses the layers below it.

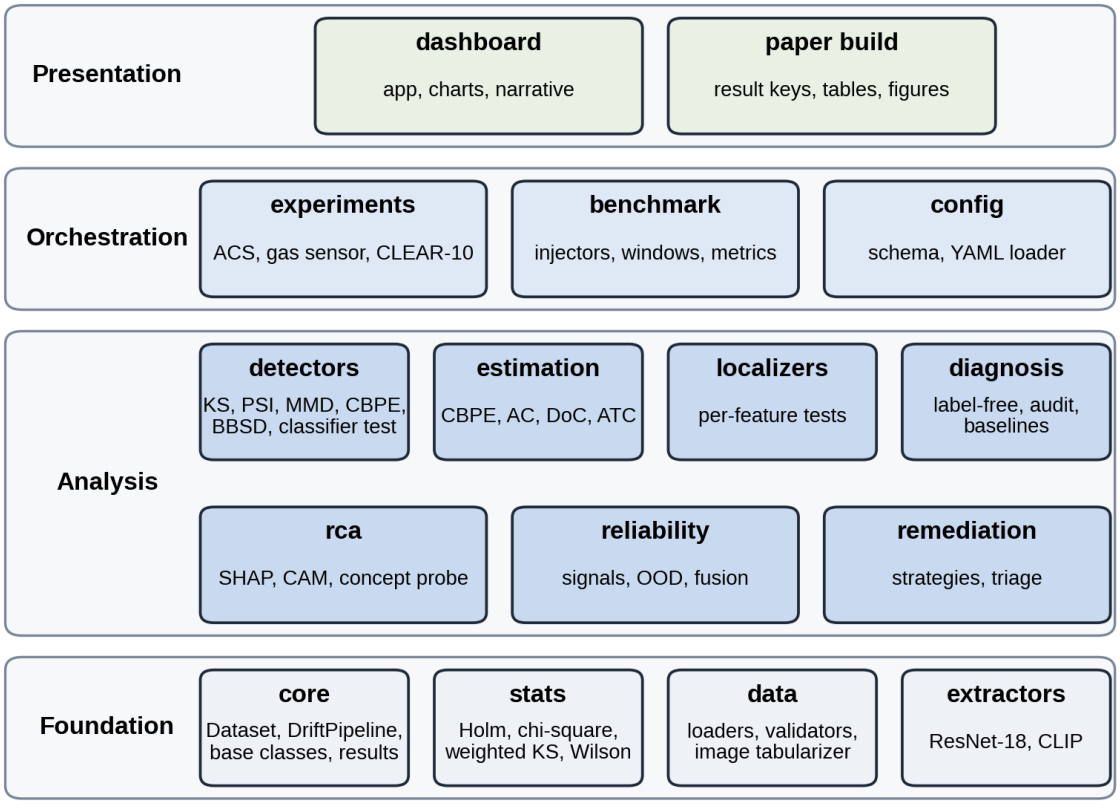


Fig. 4.2: Packages of the implementation, arranged in layers; each layer uses the layers below it

The foundation layer holds the data container and base classes (core), the statistical routines shared by every test (stats), the data loaders and validators (data) and the image networks (extractors). The analysis layer holds one package per stage of the block diagram. The orchestration layer runs studies on real data (experiments) and on injected drift (benchmark) and reads the run configuration (config). The presentation layer is the dashboard and the build of the paper. Table 4.1 gives the responsibility, inputs and outputs of each package.

Table 4.1: Responsibilities, inputs and outputs of the packages

Package	Responsibility	Inputs	Outputs
core	Dataset container with feature types; base classes for detectors, localizers and root cause analyzers; the detection pipeline; result records	Arrays or data frames, model outputs	Dataset objects, pipeline results
data	Loading and caching of the census, gas sensor and image data; schema validation; image tabularization	Raw files, Folktables downloads	Typed tables, embedding tables
extractors	Frozen ResNet-18 and CLIP image networks	Images	512-dimensional embeddings
stats	Holm and weighted Holm procedures, chi-square homogeneity, weighted Kolmogorov–Smirnov, Wilson intervals, Kish effective sample size	p values, samples, weights	Test results, intervals
detectors	Kolmogorov–Smirnov, population stability index, maximum mean discrepancy, confidence test, black-box shift detection, domain classifier	Reference and window data	Detection results with p values
estimation	Accuracy estimates from confidence: CBPE, average confidence, difference of confidences, thresholded confidence	Calibration and window probabilities	Estimated accuracy
localizers	Per-feature tests with effect sizes and unseen categories	Reference and window data	Ranked shifted features
diagnosis	Label-free stage, audit stage, combination into a verdict; rule baselines and the meta-classifier	Test p values, predictions, audit labels	Drift type, stage, rationale
rca	SHAP comparison for tabular models, class activation maps and a CLIP concept probe for images	Model, reference and window data	Attribution changes, heat maps
reliability	Per-prediction signals, out-of-distribution score, learned fusion	Calibration and window data	Risk score per prediction
remediation	Repair strategies and the label-free triage	Reference data, window data, earlier labels	Recommendation, predicted gain
benchmark	Drift injectors, window construction, grading metrics	Reference data, design, seed	Graded benchmark windows
experiments	Studies on ACS, gas sensor and CLEAR-10 data	Data, configuration	One JSON report per study
dashboard	Streamlit application, charts and generated plain-language notes	JSON reports	Interactive pages

4.3 Detailed Design

The detailed design covers the main classes and their relations, the flow of data from the public datasets to the dashboard and this report, the structure of the reports and the statistical design that every decision follows.

4.3.1 Class Design

Fig. 4.3 shows the main classes and functions in the notation of a class diagram.

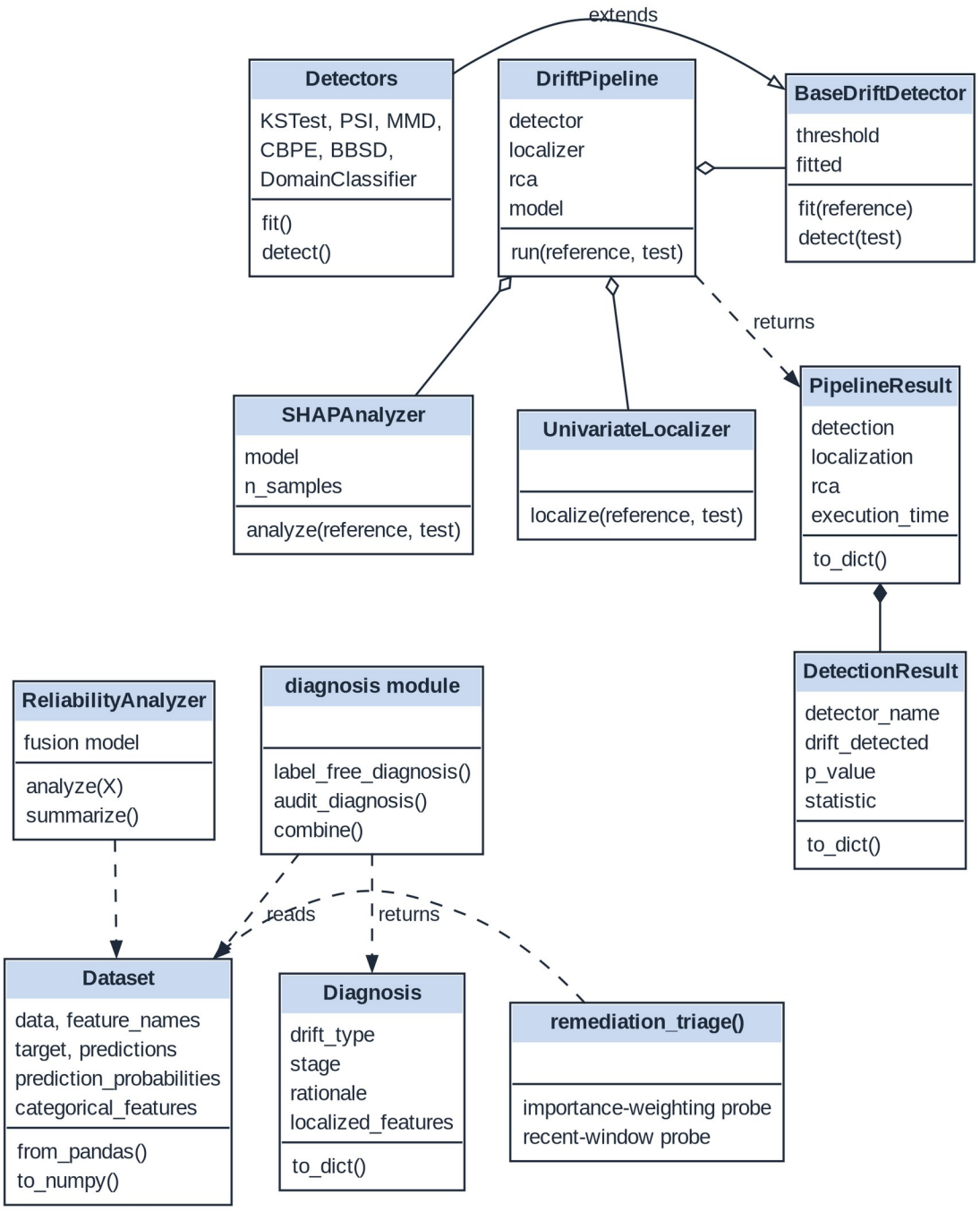


Fig. 4.3: Main classes and functions. Hollow triangles mark inheritance, diamonds mark composition and dashed arrows mark use or return values

`Dataset` is the common currency of the package: it holds the features, the optional target, the model's predictions and probabilities and the list of nominal features. Every detector extends `BaseDriftDetector`, so each offers `fit` on the reference data and `detect` on a window and

returns a `DetectionResult` with its p value and statistic. `DriftPipeline` combines a detector with an optional localizer and root cause analyzer and returns a `PipelineResult`. The diagnosis is written as plain functions rather than classes, because each step is a stateless test: `label_free_diagnosis` and `audit_diagnosis` return their evidence and `combine` turns it into a `Diagnosis` with a drift type, the stage that decided it and a rationale. `ReliabilityAnalyzer` and `remediation_triage` act on single predictions and on the decision to retrain.

4.3.2 Data Flow

Fig. 4.4 shows how data move from the public sources to the outputs of the project.

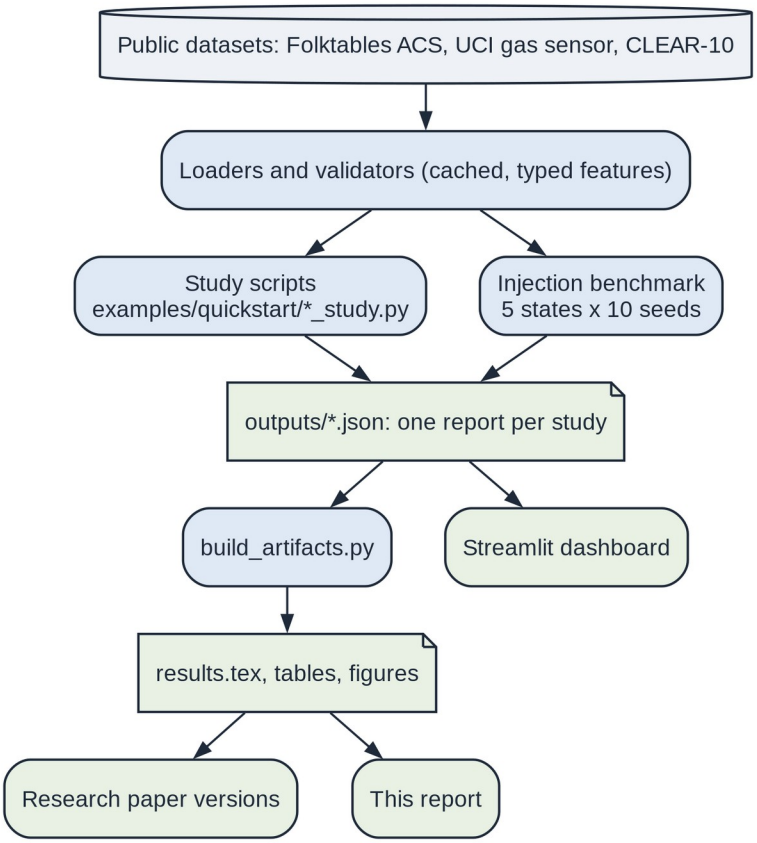


Fig. 4.4: Flow of data from the public datasets to the dashboard, the paper and this report

Loaders download the public datasets once, cache them and give every feature its type. The study scripts and the injection benchmark run the pipeline and write one JSON report per study. The dashboard reads these reports directly. A separate build script turns the same reports into a file of named result values, the tables and the figures that the paper and this report use, so that no number is ever typed by hand. If a study is run again, every number in the documents changes with it.

4.3.3 Report Structure

Each study report has four parts: the run configuration, a description of the reference data, the fitted reliability model and one record per window. A window record holds the window's size and accuracy, the result of every detector, the localization and root cause analysis, the accuracy estimates with their errors, the diagnosis with its evidence, a sample of reliability records and the time each stage took. Benchmark reports hold one record per injected window with the true drift type and intensity, so that every method can be graded on the same windows. The full schema is documented with the code in the project repository.

4.3.4 Statistical Design

Every decision in the system is a hypothesis test at the significance level $\alpha = 0.05$, with one exception: the population stability index raises an alarm at its conventional threshold of 0.2. Where many tests are made together, for example one per feature, the chance of at least one false alarm is held at α by the Holm procedure [42], its weighted form or the Bonferroni bound. The diagnosis has two stages, each tested at α, so a window without drift receives a drift verdict with probability at most 2α. Fixing these rates before the data are seen is what lets a monitoring team trust an alarm. Chapter 5 gives the tests themselves.

4.4 Project Scheduling and Tracking

The project ran over two semesters: Capstone Project I in the spring semester of 2026 and Capstone Project II in the autumn semester. Fig. 4.5 shows the timeline. The end of each phase is taken from the commit history of the code and paper repositories and its start is the team's estimate.

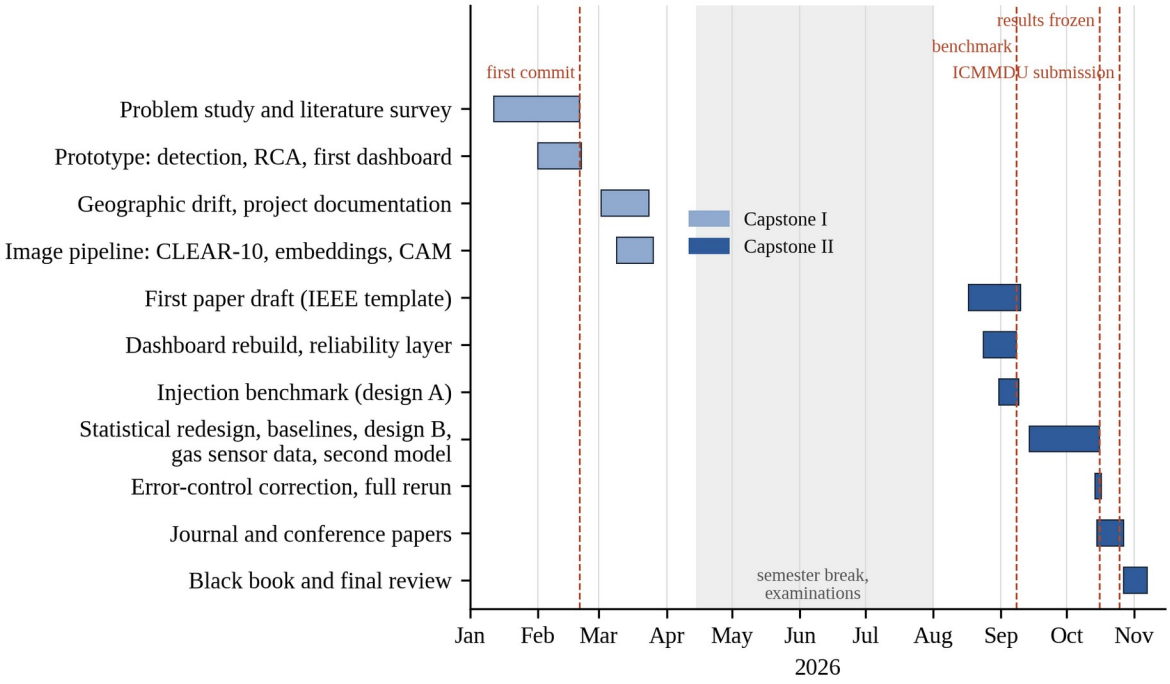


Fig. 4.5: Timeline of the project. Bars show the phases, dashed lines the main milestones and the gray band the semester break

The first semester produced a working prototype: detection with root cause analysis and a first dashboard, then geographic drift and the image pipeline on CLEAR-10. The second semester turned the prototype into a research system. The dashboard was rebuilt as a step-by-step story with the reliability layer and the injection benchmark was added. In October the diagnosis was redesigned around hypothesis tests with a stated error rate, a second model family, held-out drift mechanisms and the gas sensor data were added and the paper was written. Table 4.2 compares the milestones with the review schedule.

Table 4.2: Milestones, planned dates and dates achieved

Milestone	Planned	Achieved	Evidence
Problem statement and literature survey approved	review date	February 2026	Synopsis, review sheet 1
Working prototype with dashboard	review date	20 February 2026	First commit of the code repository
Image pipeline (CLEAR-10)	review date	25 March 2026	Commit history
Injection benchmark	review date	8 September 2026	Commit history
Error-controlled diagnosis, held-out benchmark, final results	review date	16 October 2026	Commit history; results regenerated
Research paper submitted	review date	submission date	Submission record (Appendix I)
Black book and final presentation	review date	final review date	This report

The largest deviation from a steady plan is the gap between March and August, when no commits were made during the semester break and examinations. The October redesign was also not planned at the start: a review of the first diagnosis showed that its false alarm rate was not controlled and fixing it properly took most of the month. A later code review found two further errors, one in the error control of the label-free stage and one in how the audit counted nominal categories. Both were fixed without looking at the results and every result was produced again, which changed the verdict in 115 of the 5,300 benchmark windows.

4.5 Sustainability-by-Design Decisions

Several decisions in the design reduce computation, labeling effort or risk and so serve the efficiency target of SDG 12 described in Section 1.7.

- **Labels only where they are needed.** The label-free stage settles most windows. Labels are asked for only in an audit of 600 rows and only when no label-free test rejects and the audit-size study (Section 7.4) shows that more labels add little.
- **Advice before retraining.** The triage predicts whether a cheap retrain will help before any retraining is done, so a team does not spend computation on a retrain that cannot recover the loss.
- **Frozen networks and cached embeddings.** Images are embedded once by a frozen pretrained network and the embeddings are cached and reused by every test; no network is trained on the image stream.
- **Light models and tests.** The monitored models are logistic regression and histogram gradient boosting and the tests are classical statistics, so the whole system runs on a desktop processor.
- **Capped parallelism.** Long runs use a fixed number of workers and one heavy job at a time, which keeps power draw and the risk of a failed run low.
- **Explanations with a cost limit.** Shapley values are computed for a sample of rows against a small background set and only for the model where they are fast to compute.
- **Results generated once.** Every number in the paper and this report is generated from the stored study outputs, so documents can be rebuilt without rerunning experiments.
- **Privacy by design.** All processing is local and uses public data, so no personal data are collected or moved.

CHAPTER 5: IMPLEMENTATION OF THE PROPOSED SYSTEM

This chapter describes how the design of Chapter 4 was built. It explains the development method first, then gives the algorithms of each module with their formulas, flowcharts and pseudocode and ends with the datasets: where they come from, how they were prepared and how they were used.

5.1 Methodology Employed for Development

The system was developed incrementally over the two semesters, one stage of the pipeline at a time. Each stage was first built as a working prototype, shown in the dashboard and discussed at a mentor review, then hardened with tests and rerun on all datasets. This order meant that a complete, if rough, system existed early and every later improvement could be judged on real outputs.

Four practices ran through the whole development.

- **A fixed protocol.** Every setting of an experiment, from the significance level to the audit size, was fixed before the evaluation that uses it. The diagnosis was frozen before the held-out benchmark design was run for the first time, so that design could not be tuned to it.
- **Generated results.** Every study writes a JSON report and a build script turns the reports into named result values, tables and figures. Documents cite results only through these names, so a number cannot be mistyped or left stale.
- **Seeded randomness.** Every random choice (data splits, permutations, injections, samples for explanations) takes a seed, so every result can be repeated exactly.
- **Tests and review.** Statistical routines were checked against reference implementations and by simulation of their error rates. A code review late in the project found two errors in the diagnosis; both were fixed without looking at the results and every result was produced again.

The code is written in Python 3.14 and organized as the package of Section 4.2, with two Git repositories: one for the code and one for the paper and this report. Long runs, such as the benchmark, were scheduled one at a time with a capped number of parallel workers.

A key implementation choice concerns the reference data. They are split into three disjoint parts, stratified by class: 60% for training, 20% for calibration and 20% for testing. The model is fitted

on the training part alone. The calibration part is the reference window for every detector and fits the accuracy estimator and the test part gives the reference accuracy and doubles as a control window on which no alarm should fire. Two model families are monitored, both untuned: logistic regression (LR) with L_2 regularization ($C=1$) on standardized numeric and one-hot encoded nominal features and histogram gradient boosting (GB) [43] from scikit-learn [39] with default settings and native splits on nominal codes.

5.2 Algorithms and Flowcharts for Modules Developed

The algorithms follow the stages of the block diagram. Algorithm 5.1 gives the overall processing of one production window and the subsections below describe each step.

Algorithm 5.1: Processing of one production window

```
Input:  model M, calibration part R, production window W, optional audit (X_a,
y_a)
Output: report for W
1  validate the schema and feature types of W against R
2  predict labels and class probabilities of M on W
3  run the detectors (KS, PSI, MMD, confidence test) of W against R
4  estimate the accuracy of M on W from its confidences (CBPE)
5  localize: test every feature, record effect sizes and unseen categories
6  verdict ← label-free diagnosis (Algorithm 5.2)
7  if verdict = no shift and an audit is available then
8      verdict ← audit diagnosis (Algorithm 5.3)
9  explain the change: SHAP for tabular models, CAM for image models
10 score the risk of error of every prediction
11 triage: would a cheap retrain help? (Algorithm 5.4)
12 write all results, with the time of each stage, to the report
```

5.2.1 Detection and Localization

Detection decides whether a window differs from the calibration part and localization says where the difference sits. Fig. 5.1 shows the flow.

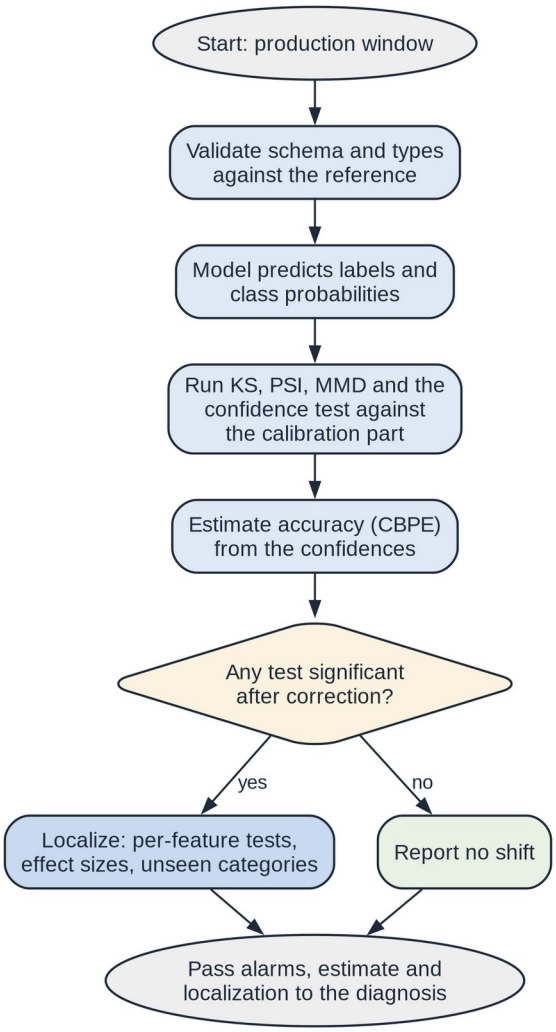


Fig. 5.1: Flowchart of detection, accuracy estimation and localization for one window

The window is validated and scored by the model, the detectors and the accuracy estimate are run and if any test is significant after correction the features are localized. The results pass to the diagnosis in either case. Four detectors are used by the diagnosis and two more act as baselines.

The **Kolmogorov–Smirnov (KS) test** compares each numeric feature. With F_P and F_Q the empirical distribution functions of the feature in the calibration part and in the window, its statistic is given by

$$D = \sup_x \left| F_P(x) - F_Q(x) \right| \tag{5.1}$$

Eq. (5.1) is evaluated for every numeric feature, with a Bonferroni correction over the features that keeps the detector's error rate at α. The **population stability index (PSI)** compares the shares p_b and q_b of the reference and the window in B bins of each feature. It is given by

$$PSI = \sum_{b=1}^{B} (q_b - p_b) \ln \frac{q_b}{p_b} \tag{5.2}$$

An alarm is raised when the index of Eq. (5.2) exceeds its conventional value of 0.2 [44]. The **maximum mean discrepancy (MMD)** test compares all standardized inputs jointly [9]. With a Gaussian kernel $k(x, y) = \exp\left(-\|x - y\|^2 / 2\sigma^2\right)$, reference rows x_1 to x_m and window rows y_1 to y_n, its statistic is given by

$$MMD^2 = \frac{1}{m^2} \sum_{i,j} k(x_i, x_j) + \frac{1}{n^2} \sum_{i,j} k(y_i, y_j) - \frac{2}{mn} \sum_{i,j} k(x_i, y_j) \tag{5.3}$$

The p value of Eq. (5.3) comes from 200 random permutations of the pooled rows. A permutation p value is given by $(1 + b)/(1 + B)$, where b of the B permutations reach the observed statistic [45]. The **confidence test** compares histograms of the model's top-class confidence c in the two samples with the chi-square test of homogeneity, whose statistic over cells with observed counts O_k and expected counts E_k is given by

$$\chi^2 = \sum_{k} \frac{(O_k - E_k)^2}{E_k} \tag{5.4}$$

Before Eq. (5.4) is evaluated, sparse cells are pooled until every expected count reaches 5 [46]. Two baselines from Rabanser et al. [3] complete the set: black-box shift detection (BBSD), which applies a KS test to each class probability. The other is a domain classifier, which trains gradient boosted trees to tell 3,000 reference rows from as many window rows and compares its held-out accuracy with $1/2$ by an exact binomial test [10].

Localization tests every feature on its own, with KS for numeric features and the chi-square test for nominal ones, under Holm correction [42]. Holm's procedure orders the m p values from smallest to largest and rejects them one by one while the i-th smallest is at most $\alpha/(m - i + 1)$, which holds the chance of any false rejection at α. Each feature also gets an effect size between 0 and 1: the KS statistic D or Cramér's V, given by

$$V = \sqrt{\frac{\chi^2}{n(k - 1)}} \tag{5.5}$$

where n is the number of rows and k the number of categories, so that Eq. (5.5) is 0 when the category shares agree and grows towards 1 as they separate. For nominal features the share of window rows whose category never occurs in the reference is reported as well, which is how new occupation codes show up.

5.2.2 Accuracy Estimation

Labels arrive late, so the accuracy is first estimated from the model's own confidence with confidence-based performance estimation (CBPE) [12]. A map $a(\cdot)$ from confidence to the probability of a correct prediction is fitted by histogram binning on the calibration part. For a window with confidences c_1 to c_n, the estimate $\hat{A}$ is given by

$$\hat{A} = \frac{1}{n} \sum_{i=1}^{n} a(c_i). \tag{5.6}$$

Three alternatives to Eq. (5.6) are fitted on the same calibration part for comparison: average confidence (AC) [22], difference of confidences (DoC) [23] and average thresholded confidence (ATC) with the negative entropy score [24]. All four assume that the link between confidence and accuracy survives the shift, so the estimate is read as an upper bound when that assumption fails.

5.2.3 Drift Type Diagnosis

The diagnosis turns an alarm into one of the four drift types and thereby into a response. Fig. 5.2 shows its decision path.

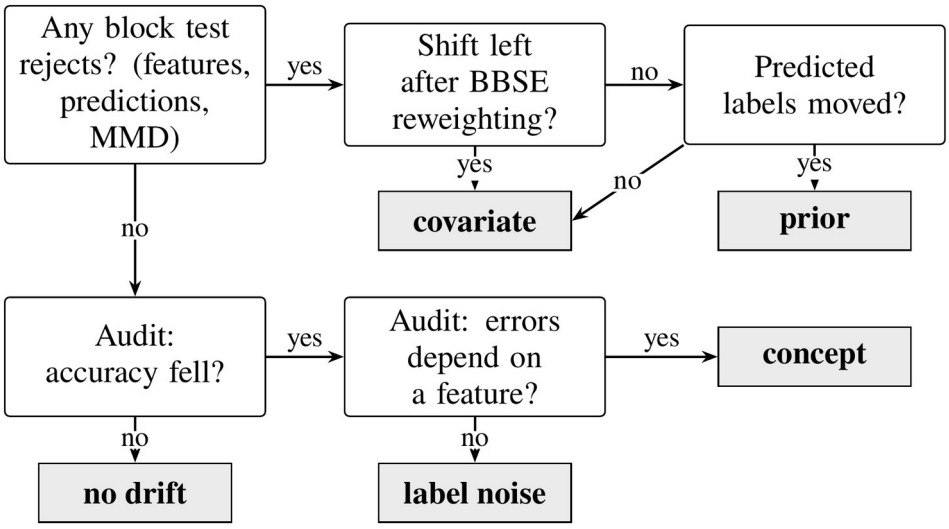


Fig. 5.2: Decision path that names the drift type. Questions are shown in white boxes and verdicts in gray boxes; the upper row needs no labels and the lower row uses the audit

The questions in the upper row use no labels and separate covariate shift from prior shift. The lower row separates concept drift from label noise and runs only when no label-free test rejects and an audit is available. Each stage is tested at level α, so a window without drift receives a drift verdict with probability at most 2α.

Label-free stage. The first question is whether anything has moved at all. Three blocks of tests answer it: the feature tests, a chi-square test on the predicted labels and the MMD test. They form one family tested at level α with a weighted Holm procedure. The weight starts out split evenly, one third per block and inside the feature block it is divided again among the individual features. A test that rejects hands its weight on to the tests still waiting, which is how the family-wise error rate survives every label-free decision. Pooling in this way matters because a permutation test cannot return a p value below $1/(B+1)$; without pooling, the MMD test would have little chance against dozens of feature tests.

A rejection raises the next question: could a change in the class balance alone produce what we see? Black-box shift estimation (BBSE) [11] first estimates the new class prior. Let $C_{jk} = P(\hat{y}=j, y=k)$ on the calibration part and let μ_j be the share of window predictions that fall in class j. The class weights $w_k = Q(Y=k)/P(Y=k)$ are given by the solution of

$$C\,w = \mu. \tag{5.7}$$

If the shift really is a pure prior shift, each class keeps its own input distribution and reweighting every reference row by the weight of its class, as given by Eq. (5.7), ought to reproduce the window's inputs. The label shift consistency test checks this, borrowing the prevalence adjustment idea of Roschewitz et al. [21]: the feature tests are repeated against the weighted reference, using Kish's effective sample size [47] and Holm correction. A feature that still differs carries a covariate shift. If no feature differs and the predicted labels have moved, the shift is a prior shift. Algorithm 5.2 gives the stage in full.

Algorithm 5.2: Label-free diagnosis

```
Input:  calibration part R (with labels and predictions), window W (predictions
only), level alpha
Output: verdict in {no shift, covariate, prior}, shifted features
1  p_f ← per-feature tests of W against R (KS or chi-square)
2  p_y ← chi-square test of the predicted labels
3  p_m ← MMD permutation test
4  weights: 1/3 per block, the feature block's third split evenly over features
5  rejected ← weighted Holm(p_f, p_y, p_m, weights, alpha)
6  if nothing is rejected then return no shift
7  w ← BBSE class weights: solve C w = mu
8  p_r ← per-feature tests of W against R weighted by w (Kish sample size)
9  residual ← features rejected by Holm(p_r, alpha)
10 if residual is not empty then return covariate, residual features
11 if the predicted labels moved then return prior
12 return covariate (joint shift seen by MMD only)
```

Audit stage. Concept drift and label noise do not touch the inputs or the model outputs, so only labels can reveal them. The audit draws m late labels (here 600) and first checks, with a one-sided two-proportion test, whether accuracy has dropped against the calibration part. If it has, the next question is whether the change in errors depends on a feature. A logistic model of whether a prediction is correct is fitted on the calibration rows and the audit rows together. Its null form holds 5 confidence bins at reference quantiles, bins of a feature f, an audit indicator and the interaction of that indicator with the confidence bins. The alternative adds the interaction of the indicator with the bins of f. A likelihood ratio test compares the two for each feature, with Holm correction across features.

The confidence term is what separates the two types. In a binary task, uniform label noise at rate r makes the probability of a correct prediction equal to

$$P(\mathit{correct} \mid x) = r + (1 - 2r)\, q(x) \tag{5.8}$$

where $q(x)$ is the model's own probability of being right at x. Under label noise the change in errors therefore depends on x only through $q(x)$, which the confidence bins of the null absorb. Concept drift, by contrast, moves the errors inside a region of some feature, so a feature that stays significant after correction points to concept drift. If no feature is significant, the verdict is label noise, as Eq. (5.8) predicts. Algorithm 5.3 gives the stage.

Algorithm 5.3: Audit diagnosis

```
Input:  model M, calibration part R with labels, audit sample (X_a, y_a), level
alpha
Output: verdict in {no drift, concept, label noise}
1  correct_R, correct_a ← whether M is right on R and on the audit
2  if the one-sided two-proportion test finds no drop in accuracy then
3      return no drift
4  for each feature f do
5      null: logit P(correct) = confidence bins + bins(f) + audit +
audit:confidence bins
6      alternative: null + audit:bins(f)
7      p_f ← likelihood ratio test of alternative against null
8  structured ← features rejected by Holm(p_f, alpha)
9  if structured is not empty then return concept, first structured feature
10 return label noise
```

5.2.4 Root Cause Analysis

The drift type suggests what kind of fix to consider but not where the trouble started. Root cause analysis asks which inputs the model now leans on more or less than before. For tabular models we compute Shapley additive explanations (SHAP) [25] of the deployed model with its

preprocessing on 1,000 rows per side against 100 background rows and compare the mean absolute attribution of every feature on reference and window rows. Each change gets a percentile bootstrap interval, Bonferroni adjusted over the features, so that only clear changes are reported. For image models, class activation maps (CAM) [28] highlight the image regions behind the prediction of the linear head on the frozen network. Fig. 5.3 shows examples from the CLEAR-10 stream.



Fig. 5.3: Class activation maps for the 4 images of CLEAR-10 bucket 10 with the highest out-of-distribution scores. The top row gives each image with its predicted and true class, the bottom row the map of evidence for the predicted class (red is most evidence)

The images were chosen as the most unusual of the bucket, because unusual images are where a change in content shows first. The model predicts all four correctly and the maps show which regions carry the evidence for each prediction; for the laptop it is the keyboard. Root cause analysis narrows the search for a cause without proving one, which is why its findings are always shown next to the localization and the drift type.

5.2.5 Reliability Scoring and Remediation Triage

The last two stages act on single predictions and on the decision to retrain. Fig. 5.4 shows both.

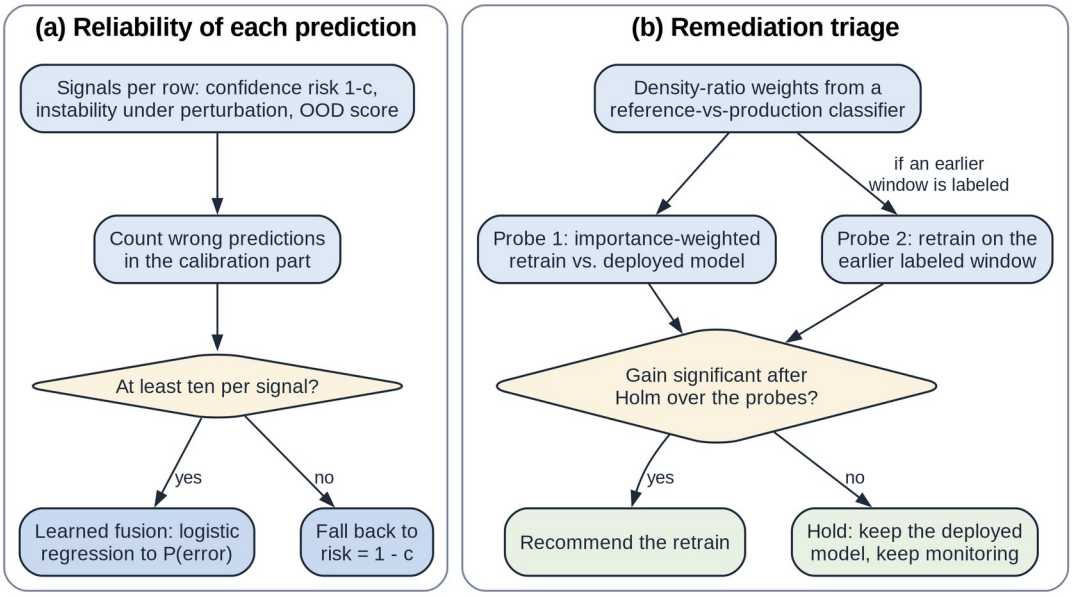


Fig. 5.4: Flowcharts of (a) the reliability score of each prediction and (b) the remediation triage

For single predictions three signals are combined, each scaled between 0 and 1: the confidence risk $1 - c$, the instability of the prediction under small perturbations of the input and an out-of-distribution (OOD) score that blends an Isolation Forest [29] with a standardized distance. A logistic regression on the calibration part maps the three signals to a probability of error. It is fitted only if the calibration part holds at least ten wrong predictions per signal [48]; with fewer, the risk falls back to $1 - c$.

The triage has to judge, without production labels, whether retraining is worth the trouble. It uses two probes, given in Algorithm 5.4. The first relies on density ratio weights from a classifier trained to tell reference rows from window rows; the weights are fitted on separate folds and capped at 20 times their mean [49]. A model retrained with them is compared with an unweighted model on a held-out 30% of the reference, weighted in the same way [50], [51]. The second probe needs late labels from an earlier window. Where they exist, it retrains on them and checks the result on a held-out part of that window.

Algorithm 5.4: Remediation triage

```
Input:  reference data R, window W (unlabeled), optional earlier labeled window
E, level alpha
Output: recommend or hold, predicted gains
1  d ← classifier separating R from W, fitted on separate folds
2  w ← density ratio weights from d, capped at 20 times their mean
3  hold out 30% of R as validation data
4  gain_1 ← weighted accuracy of the weighted retrain minus the deployed model
5  if E exists then
6       gain_2 ← accuracy on held-out E of a retrain on E minus the deployed
model
7  p ← bootstrap p values of the gains
8  if some gain is significant after Holm over the probes then recommend
9  else hold
```

5.3 Dataset Sources and Utilization

The real datasets were chosen because the cause of their drift is documented, which lets the verdicts be checked against known events. Table 5.1 lists all four and the paragraphs that follow give their source, licence, size, features, preparation and use.

Table 5.1: Datasets, drift windows and reference accuracy

						Ref. accuracy	
Dataset	**Reference**	**Windows**	**Rows**	**Inputs**	**Classes**	**LR**	**GB**
ACS Income	CA 2014	CA 2015–19 and 2021–23; TX, NY, FL, WA 2014	36,162–203,278	10	2	0.826	0.833
ACS Employment	CA 2014	CA 2015–19 and 2021–23; TX, NY, FL, WA 2014	56,576–326,241	16	2	0.729	0.772
Gas sensor	batches 1–2	batches 3–10	161–3,613	128	6	0.982	0.985
CLEAR-10	bucket 1	buckets 2–10	3,297–3,300	512	11	0.914	0.897

Rows gives the smallest and the largest window, Inputs the number of features or embedding dimensions and the last two columns the reference test accuracy of LR and GB.

ACS Income and ACS Employment. Both tasks come from the American Community Survey (ACS) of the United States Census Bureau, as packaged by Folktables [26] (code under the MIT licence; data under the Census Bureau’s terms of use; accessed through the Folktables package). ACS Income predicts from 10 features whether a person earns more than $50,000 and ACS Employment predicts from 16 features whether a person aged 16 to 90 is employed. Table 5.2 lists the features of ACS Income. California (CA) 2014 is the reference, with 110,364 training rows for ACS Income. The temporal windows are CA 2015 to 2019 and 2021 to 2023, as no standard 2020 release exists [52] and the geographic windows are Texas (TX), New York (NY), Florida (FL) and

Washington (WA) in 2014. Missing values are coded as -1. In 2019 the relationship variable RELP was replaced by RELSHIPP [53], which we map back to RELP with the code labels of the 2018 and 2019 data dictionaries. Five documented changes fall in these years: new occupation codes in 2018 [54], a redesigned relationship question in 2019, a new coding of race answers from 2020 [55], the pandemic and wage growth against the fixed income threshold.

Table 5.2: Input features of ACS Income

Code	Description	Type
AGEP	Age	Numeric
COW	Class of worker	Nominal
SCHL	Educational attainment (24 ordered levels)	Ordinal, treated as numeric
MAR	Marital status	Nominal
OCCP	Occupation	Nominal
POBP	Place of birth	Nominal
RELP	Relationship to the reference person	Nominal
WKHP	Usual hours worked per week	Numeric
SEX	Sex	Nominal
RAC1P	Race	Nominal

Nominal codes such as occupation are one-hot encoded for LR, split natively by GB and tested with count-based statistics by the detectors, because their numeric order means nothing.

Gas sensor array drift. This dataset from the machine learning repository of the University of California, Irvine (UCI) holds 13,910 measurements of six gases by 16 metal oxide sensors, taken over 36 months in ten batches [56], [57], with eight features per sensor (licence CC BY 4.0). As the sensors age, their response to the same gas drifts while the labels stay exact, a documented change in $P(X \mid Y)$. Batches 1 and 2 form the reference and batches 3 to 10 the windows. Some batches are small, so the number of audit labels for a window of n rows is given by $min(600, \lfloor n/2 \rfloor)$.

CLEAR-10. This image stream from Flickr has ten object classes and a background class, spread over ten time-ordered buckets of about 3,300 images each [27] (licence CC BY 4.0). Bucket 1 is the reference. Images are mapped to the 512-dimensional output of the global average pooling layer of a frozen residual network with 18 layers (ResNet-18) trained on ImageNet [58] and both models are fitted on these embeddings.

Injection benchmark. The type of a real drift is never known for certain, so the diagnosis is graded on drift that we inject ourselves into real census data. The benchmark uses the 2014 data

of 5 states (CA, TX, NY, FL, WA) for both ACS tasks. For each of 10 seeds the split, the window and the injection are drawn again, which gives 50 replicates per configuration. A replicate draws a window of 4,000 rows and a grading holdout of 2,000 rows from the test part and each mechanism is injected into both at three intensities. Table 5.3 lists the mechanisms of the two designs.

Table 5.3: Drift mechanisms of the injection benchmark

Type	Design A (partly matches the tests)	Design B (held out from the design)
Covariate shift	Rescale two numeric features (ACS Income: AGEP and WKHP) for a random subset of rows	Selection bias: keep each row with probability $\sigma(\beta s(x))$ for a seeded random score s over all inputs, $\beta \in \{0.5, 1, 1.5\}$
Prior shift	Drop rows of one class until the positive rate rises by 0.14, 0.23 and 0.32	Lower the positive rate by 0.08, 0.14 and 0.20
Concept drift	Invert the labels of the 15, 30 and 50% of rows with the highest value of one feature (ACS Income: SCHL)	Invert the labels in the top 15, 30 and 50% of a second random score (an oblique region); for ACS Income also raise the income threshold to $55,000, $60,000 and $70,000
Label noise	Flip a random 8, 18 and 32% of the labels	Flip positive labels to negative at rates of 10, 20 and 35%
Control	A window without drift in every replicate	None

In design B the rows of the selection-bias windows stay real and $P(Y \mid X)$ is exactly unchanged, so the mechanism is a pure covariate shift that moves many features a little rather than two features a lot. None of the design B mechanisms shaped any part of the procedure, which is what makes it a fair test of whether the diagnosis carries over to new drift.

CHAPTER 6: TESTING OF THE PROPOSED SYSTEM

A monitoring system makes claims about error rates, so testing it means more than checking that the code runs. This chapter states what the testing had to establish, describes the kinds of test used, lists the test cases with their results and draws conclusions from them.

6.1 Introduction to Testing

The testing had three objectives. The first was functional: every module must do what its requirement in Section 3.2 says. The second was statistical: every test must hold its stated error rate and the diagnosis must name the drift type correctly when the truth is known. The third concerned the system as a whole: the full pipeline must behave sensibly on real data and the dashboard must present every study without failing.

The scope covers the Python package, the study scripts, the benchmark and the dashboard. All tests ran on the workstation of Section 3.4 under Python 3.14 with pytest 9.0. The strategy was layered: unit tests for each routine, simulations of error rates for the statistical procedures, the injection benchmark for the diagnosis, real-data studies with control windows for the whole system and integration tests for the dashboard. The automated suite was run again on 30 October 2026 for this report; its output is kept in the project folder.

6.2 Types of Tests Considered

Each kind of test below was chosen because it checks a different claim the system makes.

- **Unit tests** (94 tests) check single routines against known answers: the Holm procedure against its step-down definition, the weighted KS test against SciPy with unit weights, the chi-square pooling rule, the BBSE weights on a known prior shift, the CBPE estimate on a labeled reference and each drift injector against the definition of its type.
- **Smoke tests** (6 tests) check that the main objects can be created and that the basic pipeline runs end to end on small data.
- **Integration tests** (13 tests) load every study report into the dashboard, build every chart and plain-language note, check that the decision path shown matches the verdicts and render the whole application without exceptions.

- **Statistical validation** checks error rates by simulation: the MMD permutation test and the weighted Holm procedure must not reject a true null more often than α and the family-wise error of the label-free decisions was simulated with 200,000 replicates.
- **Model validation** grades the diagnosis on the injection benchmark, where the true type is known, against rule baselines, a learned meta-classifier and ablations of the procedure (Section 7.4).
- **System tests on real data** run the whole pipeline on the four real datasets, where the control windows must stay quiet and the verdicts must agree with the documented causes.
- **Performance tests** record the time each stage takes on every window.
- **Reproducibility tests** check that seeded components (permutation tests, SHAP, gradient boosting, strategies) give identical results when run twice and that the GPU and CPU versions of the permutation tests agree.

Security and load tests were not considered relevant: the system runs locally on public data, serves no external users and processes batches rather than requests.

6.3 Test Case Scenarios

Table 6.1 lists the main test case scenarios. Each is tied to a requirement of Chapter 3 and the last column gives the outcome with the test or result that supports it.

Table 6.1: Test case scenarios and their results

ID	Requirement	Input / precondition	Expected result	Actual result	Status / evidence
TC-01	FR-01	Dataset with a target of the wrong length; invalid detector options	Rejected with a clear error	Rejected	Pass; test_dataset_and_detector_options_are_validated
TC-02	FR-01	Nominal and numeric features in one window	KS skips nominal features; localizer uses chi-square for them	As expected	Pass; test_ks_skips_categorical_and_localizer_uses_chi2
TC-03	FR-04	Two samples from the same distribution, many repetitions	MMD rejects in at most about α of runs	Within α	Pass; test_mmd_permutation_test_controls_type_one_error
TC-04	FR-04	One large-scale feature hiding a shift in a small one	Standardized MMD still detects the shift	Detected	Pass; test_mmd_standardize_exposes_shift_hidden_by_large_scale_feature

ID	Requirement	Input / precondition	Expected result	Actual result	Status / evidence
TC-05	FR-04	Production values outside the reference range	PSI counts them instead of dropping them	Counted	Pass; test_psi_counts_values_outside_reference_range
TC-06	FR-05	Labeled reference with known accuracy	CBPE estimate close to the true accuracy	Close	Pass; test_cbpe_estimates_accuracy_from_labelled_reference
TC-07	FR-06	Planted shift in one feature	Only that feature is flagged after Holm correction	Only that feature	Pass; test_univariate_localizer
TC-08	FR-07, NFR-03	Several true nulls and one shifted feature	Weighted Holm with equal weights equals Holm; family-wise error at most α	Equal; error at most α	Pass; test_weighted_holm_equal_weights_is_holm_and_controls_fwer; simulation 4.9%
TC-09	FR-07	Pure prior shift injected	BBSE recovers the new prior; verdict prior shift	Recovered; prior	Pass; test_bbse_recovers_a_prior_shift
TC-10	FR-07, FR-08	One window of each injected type and one control	Each type named correctly	All five named	Pass; test_diagnosis_names_each_injected_type
TC-11	FR-08	Audit with nominal categories, some absent from the audit	Bins counted on the audit rows only	As expected	Pass; test_audit_bins_count_categories_in_the_audit_only
TC-12	FR-09	Model with real change and pure noise	SHAP intervals flag the real change only	As expected	Pass; test_shap_change_intervals_separate_noise_from_real_change
TC-13	FR-10	Calibration part with too few wrong predictions	Fusion falls back to confidence risk	Falls back	Pass; test_fusion_falls_back_without_enough_errors
TC-14	FR-11	Correctable covariate shift; concept shift	Triage escalates on the first and holds on the second	As expected	Pass; test_triage_escalates_on_correctable_covariate_shift_and_iw_recovers, test_triage_holds_on_concept_shift
TC-15	FR-12, FR-13	Every study report	Dashboard loads all reports, draws every chart and renders without exceptions	As expected	Pass; 13 integration tests
TC-16	FR-14	Each injector applied to real rows	Each injector changes only what its type allows	As expected	Pass; five injector tests
TC-17	NFR-01	200 benchmark control windows	Each detector’s false alarm CI contains α or lies below it	As expected	Pass; Section 7.4

ID	Requirement	Input / precondition	Expected result	Actual result	Status / evidence
TC-18	NFR-02	Benchmark control windows through the full diagnosis	Drift verdicts in at most $2\alpha = 10\%$	6.5%	Pass; Section 7.4
TC-19	System	Real control windows of all four datasets	No drift verdict on any control window	None	Pass; Section 7.4.5
TC-20	NFR-04	Every real window	Analysis within a daily monitoring cycle	At most 90 s	Pass; Section 7.4.7
TC-21	NFR-06	Seeded components run twice; GPU against CPU	Identical results	Identical	Pass; test_strategies_are_deterministic, test_gpu_permutation_tests_match_cpu
TC-22	NFR-07	Full automated suite	All tests pass	113 passed in 49 s	Pass; pytest run of 30 October 2026

6.4 Inference Drawn from the Test Cases

All 113 automated tests pass and every test case in Table 6.1 met its expected result. The statistical tests matter most, because they back the error rates the system promises. The family-wise error of the label-free decisions came out at 4.9% against a target of 5%. The rule used before the correction, which tested the blocks first and then the features at the level the block was rejected at, came out at 6.6%: it spent α twice. On the benchmark control windows the full diagnosis raised a false verdict on 6.5% of windows, below its bound of 10%.

Testing also found defects. The simulation above exposed the error in the label-free stage and a code review found that the audit counted the categories of nominal features on all rows instead of the audit rows only. Both were fixed, a regression test was added for each (TC-08 and TC-11) and every result was produced again, which changed the verdict in 115 of the 5,300 benchmark windows. Earlier fixes are guarded in the same way: tests now check that SHAP explains the positive-class probability of the full model pipeline, that the MMD test standardizes its inputs, that CBPE stays finite when a confidence bin of the reference is empty and that permutation p values are seeded.

Two limits remain. The automated suite checks behavior on small and synthetic data and does not replace the benchmark and real-data studies, which take hours rather than seconds. And usability was checked by the team and the mentor reviews, not by a study with outside users, so the usability requirement is met only in the sense of NFR-08.

CHAPTER 7: RESULTS AND DISCUSSION

This chapter presents what the finished system does and how well it does it. It starts with the dashboard, through which a user meets every module, then defines the measures used to judge the system, lists its inputs and settings, reports the results on the benchmark and on real drift, compares them with existing approaches and draws conclusions. It ends with the sustainability assessment that the department asks for.

7.1 User Interface Screenshots for Respective Modules

The dashboard is a Streamlit application with one tab per dataset (ACS census data, CLEAR-10 images, the gas sensor array) and one for the injection benchmark. Each tab tells the story of a study in numbered steps that follow the stages of the pipeline and each step opens with the question it answers and ends with a short note, generated from the results, that says what the data show. The screenshots below were taken from the running dashboard on the ACS Income study with logistic regression unless stated otherwise. Fig. 7.1 shows the top of the ACS tab.

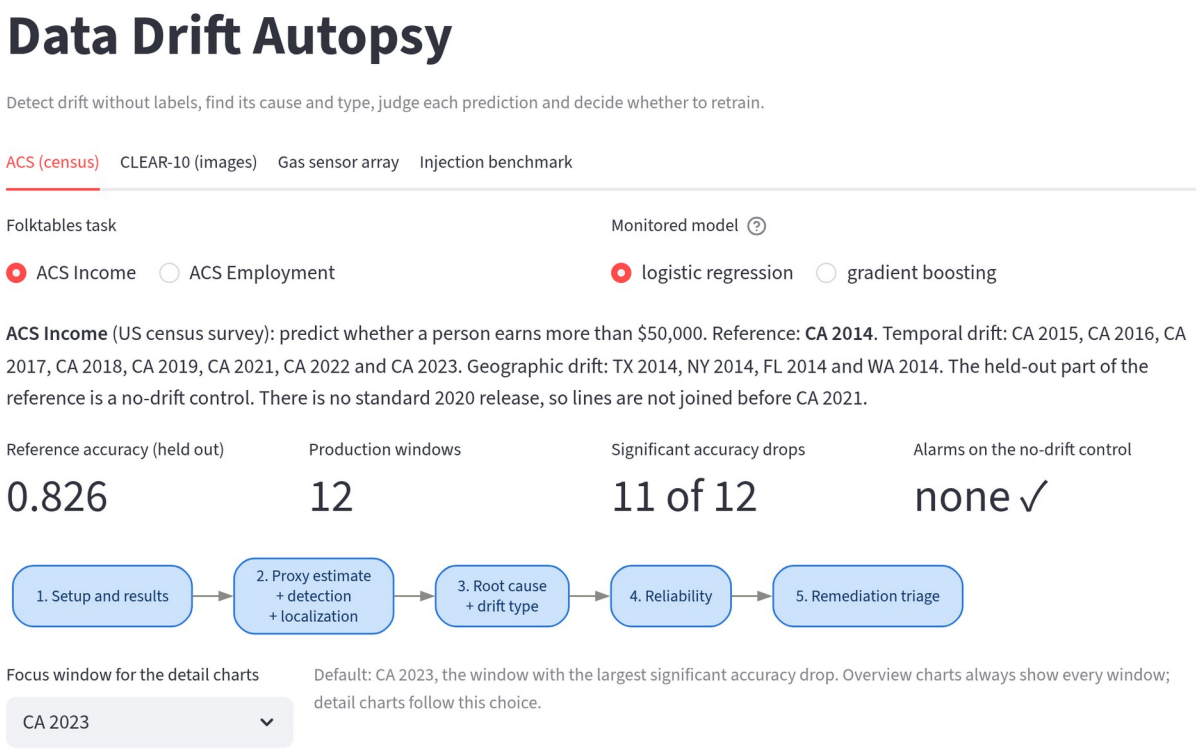


Fig. 7.1: Top of the ACS tab: task and model selectors, the description of the study, the headline numbers and the strip of steps

The user chooses the task (ACS Income or ACS Employment) and the monitored model. The headline numbers summarize the study: the reference accuracy of 0.826, the 12 production windows, the 11 windows whose accuracy dropped significantly and the absence of any alarm on the control window. The strip of steps doubles as a table of contents. Fig. 7.2 shows the first analysis step, the label-free accuracy estimate.



Fig. 7.2: Step 2a: the label-free accuracy estimate (CBPE) against the measured accuracy, its error per window and the confidence histograms of the reference and of CA 2023

The left chart shows that the estimate follows the measured accuracy closely at first and then stays too high as the years pass and the bar chart on the right gives the size of that error per window. The confidence histograms below show why: the model's confidence barely moves in CA 2023

even though its accuracy has fallen, which is the failure mode described in Section 5.2.2. Fig. 7.3 shows how each detector behaves on the same windows.

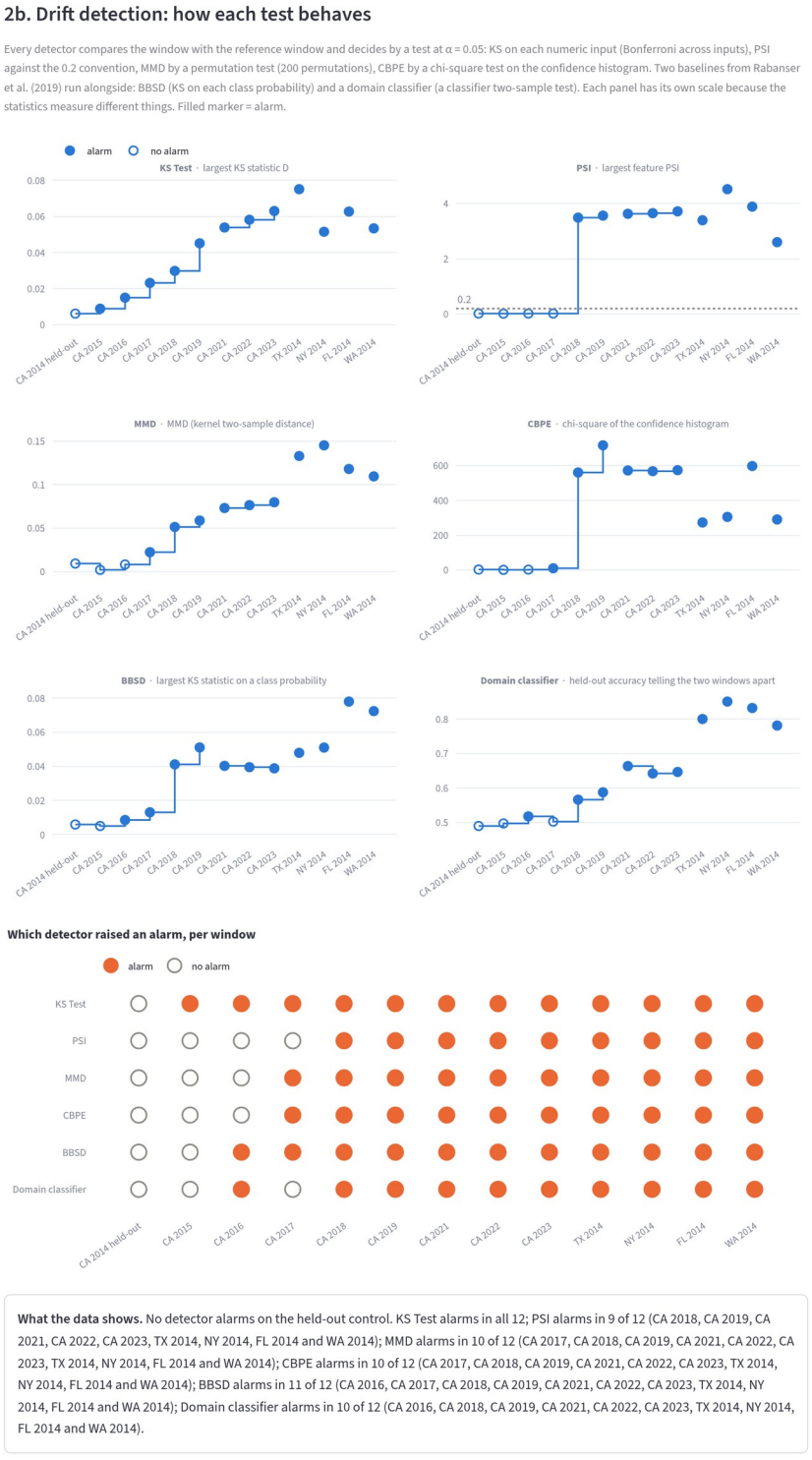


Fig. 7.3: Step 2b: the statistic of every detector per window and, at the bottom, which detector raised an alarm on which window (filled circles)

Each small chart traces one detector's statistic across the windows and the alarm matrix at the bottom shows at a glance that no detector fires on the held-out control window while most fire from 2016 onwards. The population stability index is the most reluctant detector. Fig. 7.4 shows where the inputs moved.

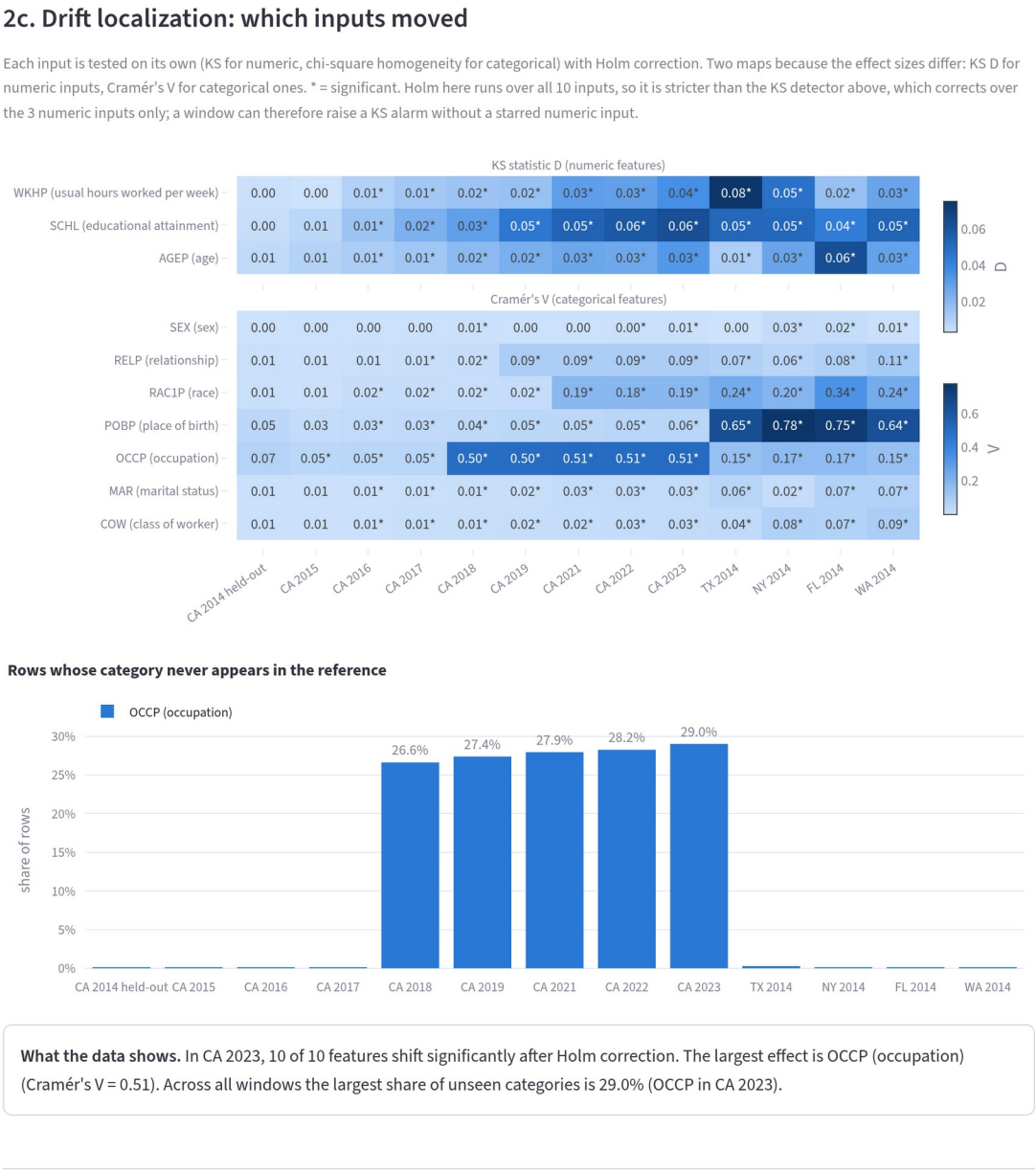


Fig. 7.4: Step 2c: effect size of every feature per window (KS statistic for numeric features, Cramér's V for nominal ones; a star marks a significant shift) and the share of rows with occupation codes never seen in the reference

The heat maps show that occupation and place of birth carry the largest shifts and the bar chart at the bottom shows the jump in unseen occupation codes from 2018, when the census introduced new codes. Fig. 7.5 shows what the model now relies on.

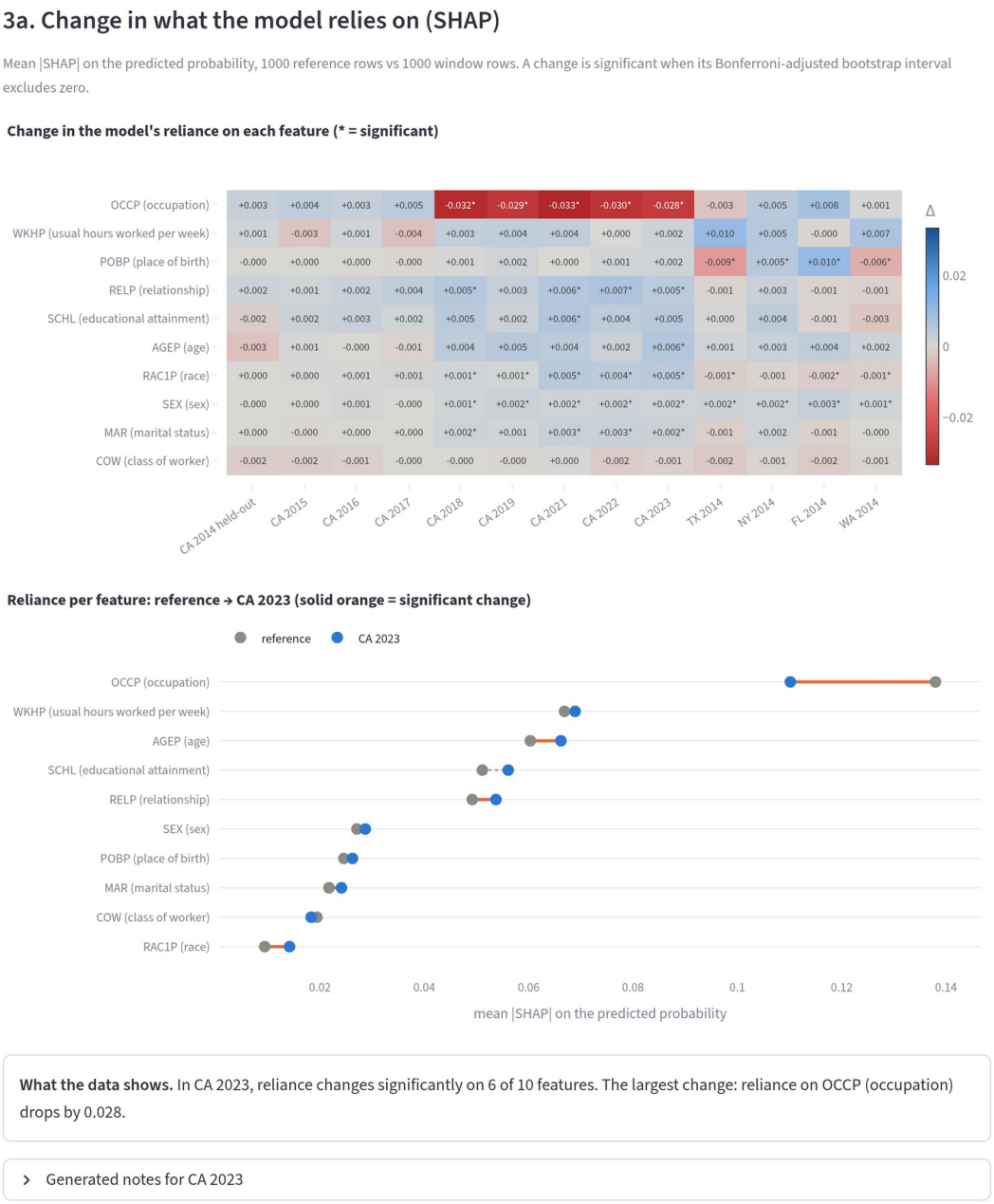


Fig. 7.5: Step 3a: change in the mean absolute SHAP value of each feature per window and the reliance on each feature in the reference and in CA 2023

The model leans less on occupation in the later windows, consistent with codes it has never seen, while its reliance on the other features changes little. Fig. 7.6 shows the drift type.

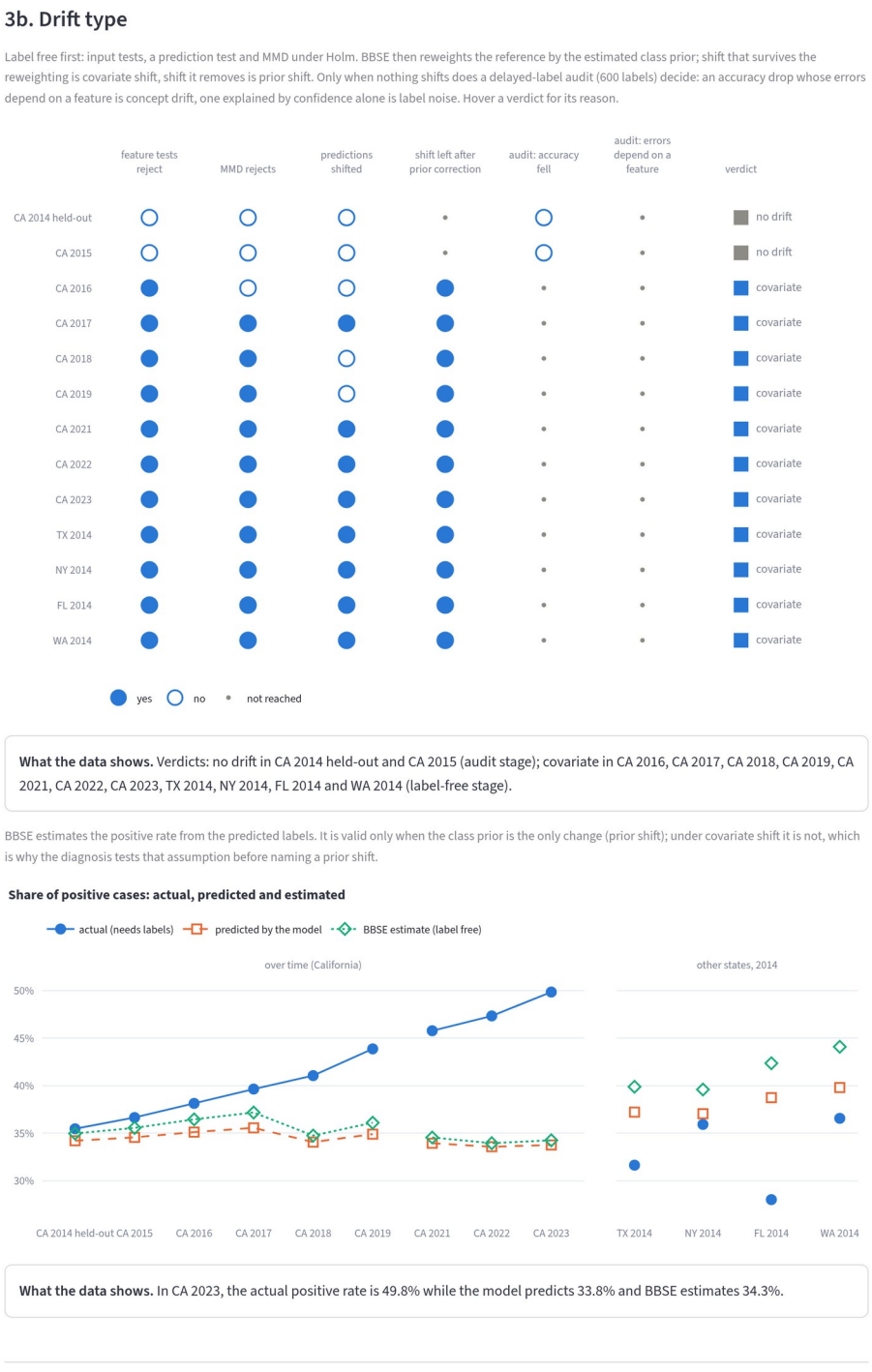


Fig. 7.6: Step 3b: the decision path of the diagnosis for every window, from the label-free tests to the audit, with the verdict on the right; below, the actual, predicted and BBSE-estimated share of positive cases

Each row is a window and each column a question of the decision path in Fig. 5.2. The control window and CA 2015 pass every label-free test and the audit, so they are called no drift, while every later window is stopped at the label-free stage as covariate shift. The chart below shows the

label problem of CA 2023: the actual share of positive cases rises much faster than the model predicts, which no label-free test can see. Fig. 7.7 shows the reliability of single predictions.

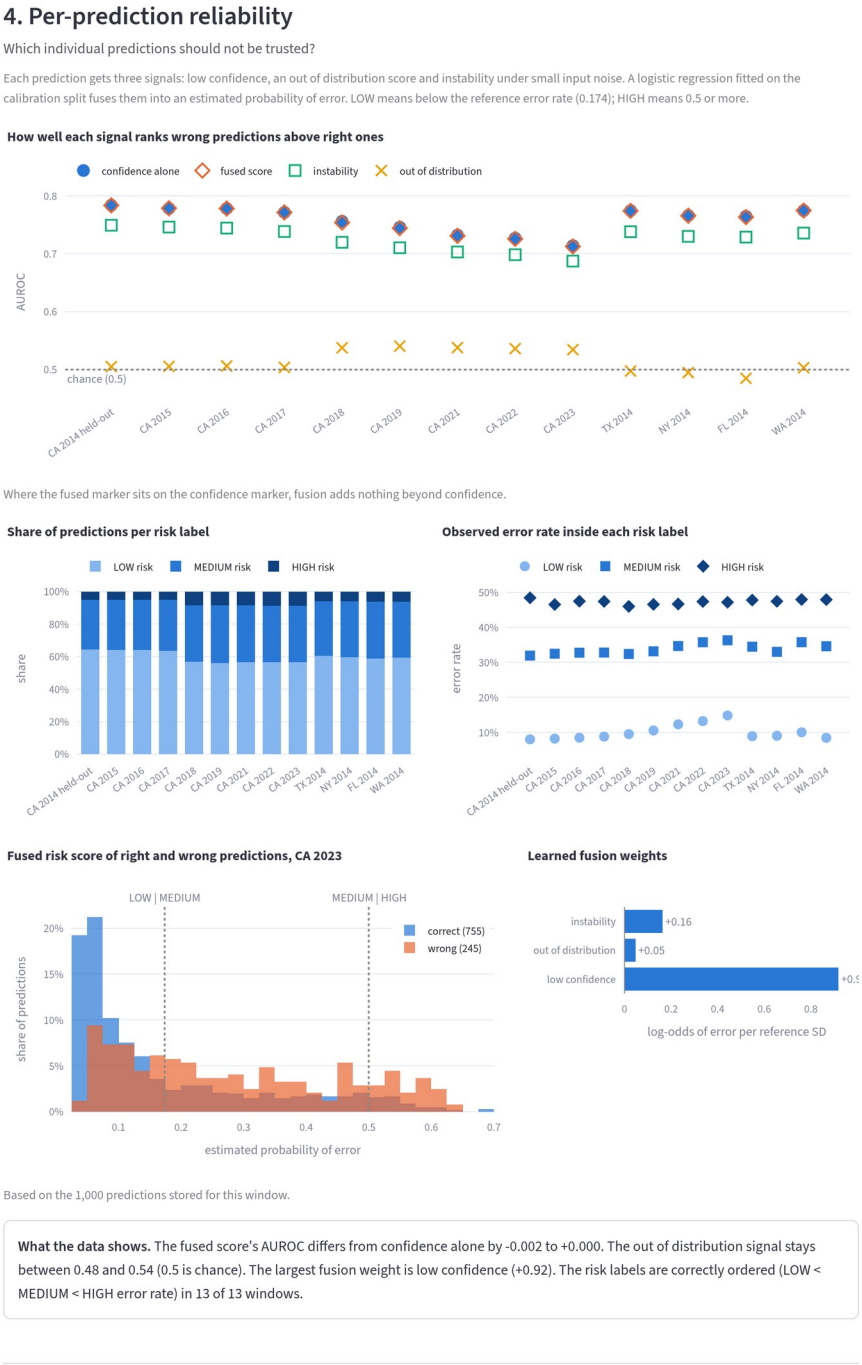


Fig. 7.7: Step 4: how well each signal ranks wrong predictions above right ones (area under the ROC curve), the share of predictions per risk label, the error rate inside each label and the weights the fusion learned

The fused score sits on top of confidence alone in every window and the learned weights put almost all the emphasis on confidence. The risk labels are ordered correctly: the high-risk group has the highest error rate. Fig. 7.8 shows the remediation step.

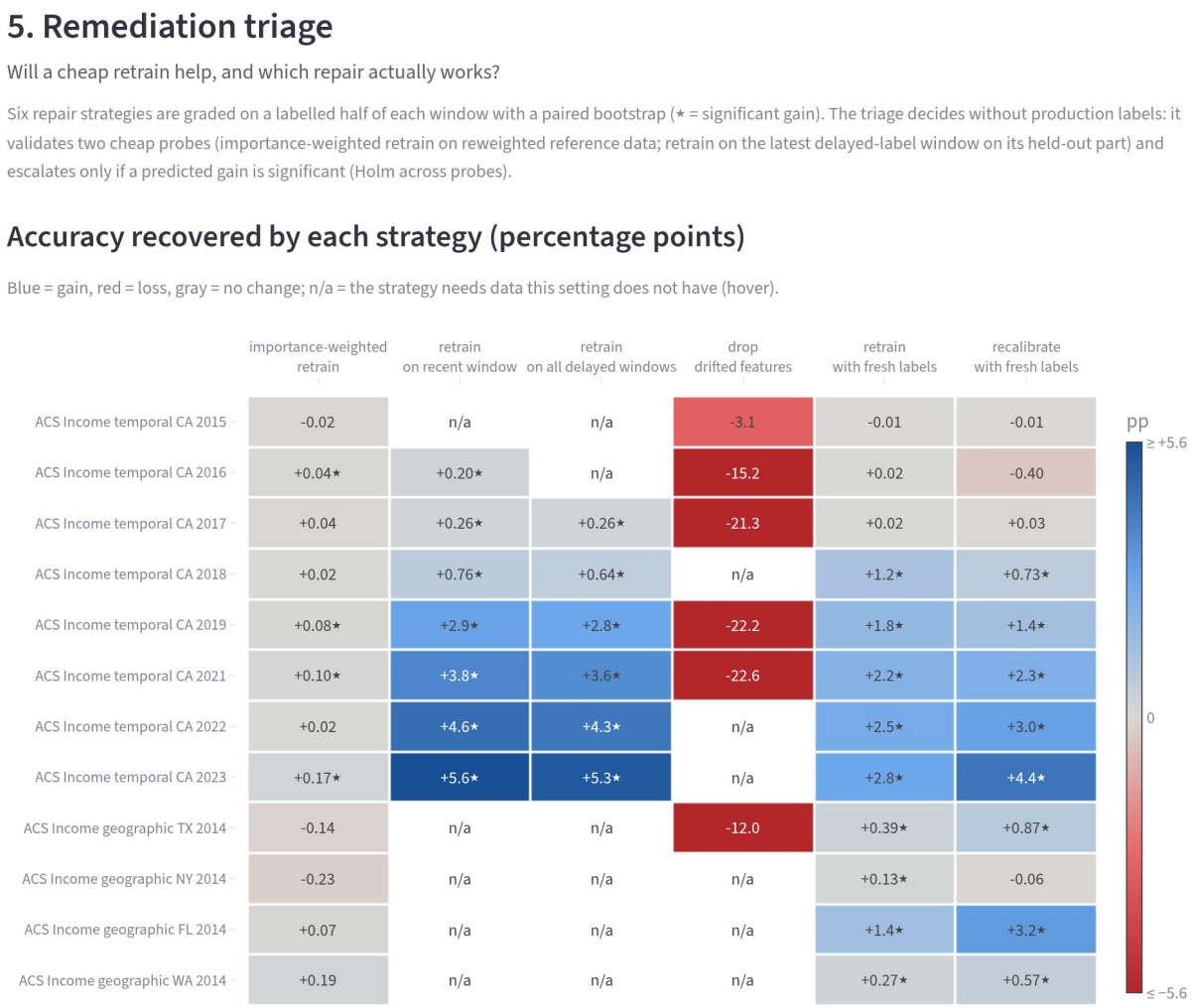


Fig. 7.8: Step 5: accuracy recovered by each repair strategy in each window, graded on labeled data (blue means a gain, red a loss, gray not applicable)

Retraining on the most recent labeled window recovers accuracy in almost every temporal window, while importance weighting helps only in some and dropping drifted features costs accuracy. Fig. 7.9 shows the image tab.

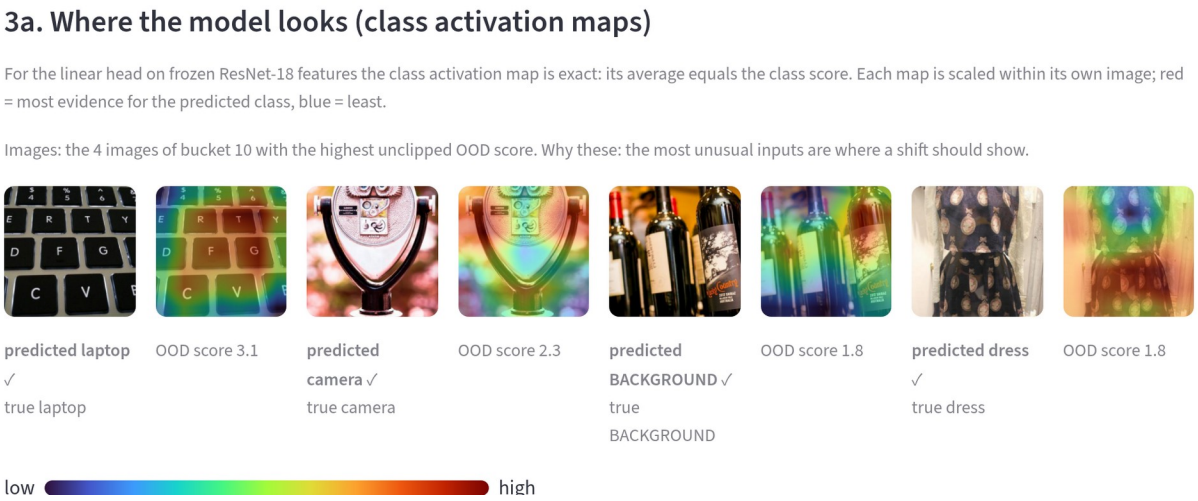


Fig. 7.9: CLEAR-10 tab, step 3a: class activation maps of the most unusual images of bucket 10

For images, root cause analysis shows where the model looks, as described in Section 5.2.4. The benchmark tab grades the diagnosis on injected drift; Fig. 7.10 shows its drift type step.

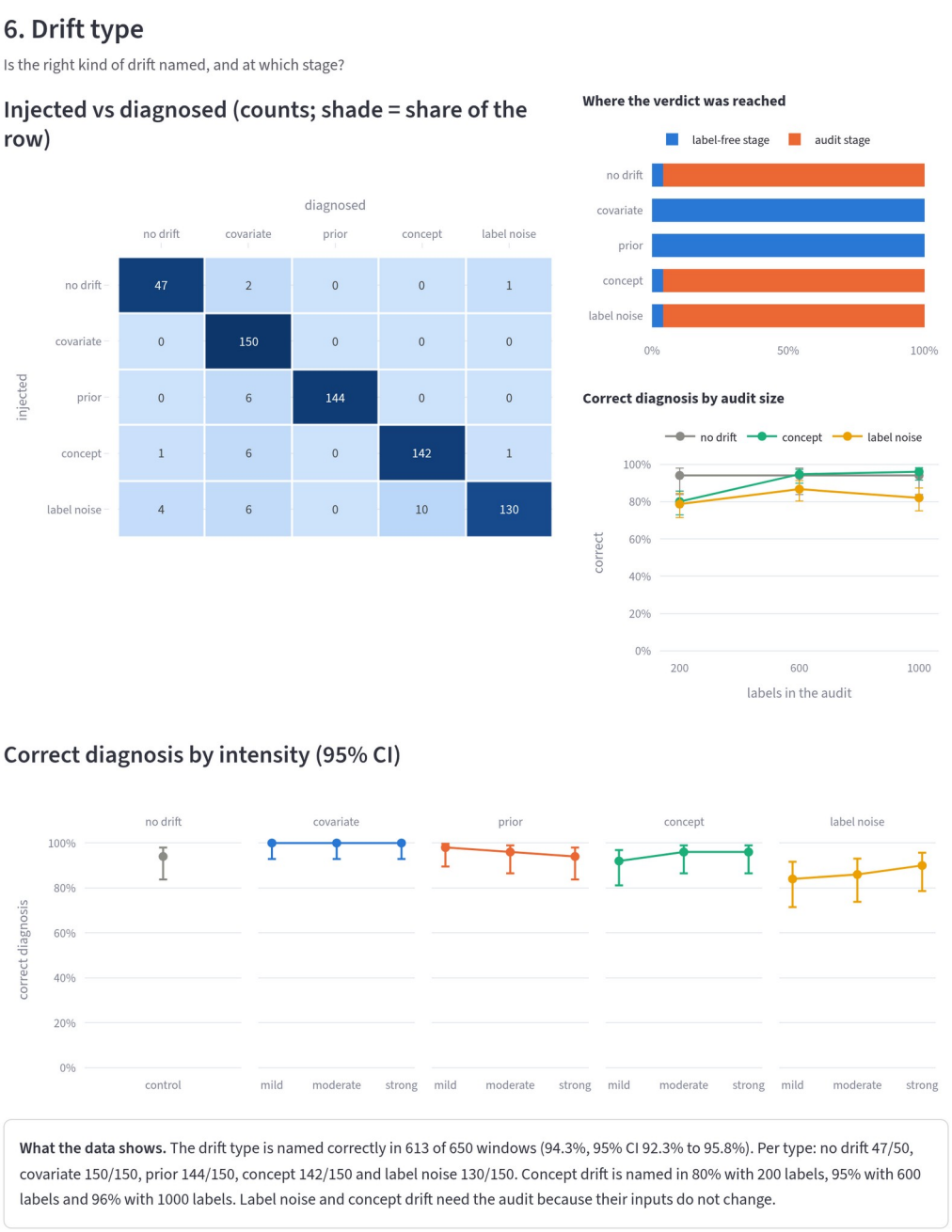


Fig. 7.10: Benchmark tab, step 6: injected against diagnosed drift types, the stage at which each verdict was reached, the effect of the audit size and the correct diagnosis by intensity

The confusion matrix fills mostly the diagonal, the stage chart shows that covariate and prior shifts are settled without labels while concept drift and label noise need the audit and the intensity chart shows that the hardest cases are the mildest ones. Fig. 7.11 compares the procedure with the alternatives.



Fig. 7.11: Benchmark tab, step 7: share of windows whose drift type is named correctly by our procedure, the rule baselines, the learned meta-classifier and the ablations, with paired tests

The chart and the table of paired tests show the comparison that Section 7.4 reports in full.

7.2 Performance Evaluation Measures

Each part of the system is judged by a measure suited to the claim it makes.

- **Correct-type rate.** The share of benchmark windows whose drift type is named correctly, with a 95% Wilson interval [59]. It is the main measure of the diagnosis because a wrong type leads to a wrong response.

- **False alarm rate.** The share of control windows (no drift) on which a detector raises an alarm or the diagnosis names a drift type. It checks the error rates the design promises: at most α for each detector and 2α for the diagnosis.
- **Paired comparison.** Each method is compared with ours through the per-replicate difference in the correct-type rate, tested by a sign-flip permutation test with 10,000 flips and Holm correction. Replicates are the unit of analysis because the windows of one replicate share a model and a reference.
- **Mean absolute error of accuracy estimates.** The average absolute difference, in percentage points, between an estimated and a measured accuracy.
- **Area under the ROC curve (AUROC).** How well a risk score ranks wrong predictions above right ones; 0.5 is chance and 1 is perfect.
- **Triage accuracy and recall.** The share of settings in which the triage's advice matches whether a retrain truly helped and the share of truly helpful retrains it recommends.
- **Run time.** Seconds per window through the stages of the pipeline, excluding model training and image embedding.

7.3 Input Parameters / Features Considered

The inputs of each study are the features of its dataset (Table 5.2 for ACS Income; 16 features for ACS Employment, 128 sensor features for the gas array and 512 embedding dimensions for CLEAR-10) and the model's predictions and class probabilities. Table 7.1 lists the settings that control the tests, all fixed before the evaluation.

Table 7.1: Settings of the pipeline and the benchmark

Setting	Value	Rationale
Significance level α	0.05	Conventional level; fixes the false alarm rate
Reference split (train / calibration / test)	60 / 20 / 20%	Disjoint parts for fitting, reference windows and control
PSI alarm threshold	0.2	Conventional value [44]
MMD permutations	200	Enough for p values well below α at modest cost
Minimum expected count per chi-square cell	5	Standard rule for the chi-square approximation [46]
Audit labels per window	600	Where more labels stop paying off (Fig. 7.13)
Confidence bins in the audit null	5	Absorb the confidence-driven part of error changes
Domain classifier rows per side	3,000	Balanced classifier test
SHAP rows per side / background rows	1,000 / 100	Stable attributions at bounded cost
Importance weight cap	20 × mean	Limits the variance of weighted estimates

Setting	Value	Rationale
Triage validation share	30%	Held-out reference for the weighted comparison
Benchmark states / seeds / replicates	5 / 10 / 50	Variation across populations and draws
Benchmark window / holdout rows	4,000 / 2,000	Realistic window size; labeled grading data

7.4 Graphical and Statistical Output

The results are presented in the order of the questions they answer: how often the drift type is named, whether that carries over to new drift, how many labels the audit needs, how well the building blocks work, how the system behaves on real drift, what the last two stages achieve and how long it all takes.

7.4.1 Drift Type against Baselines

The most direct test of the diagnosis is how often it names the injected type, compared with the alternatives. Table 7.2 gives this share for every configuration, each with 50 replicates.

Table 7.2: Share of benchmark windows with the correct drift type (%)

	Logistic regression				Gradient boosting			
Method	**Inc. A**	**Inc. B**	**Emp. A**	**Emp. B**	**Inc. A**	**Inc. B**	**Emp. A**	**Emp. B**
Ours (hierarchical tests)	94.3	62.4	85.7	80.5	94.2	63.2	90.5	81.5
Input monitoring	30.3 ↓	20.0 ↓	30.5↓	25.0↓	30.3 ↓	20.0 ↓	30.5↓	25.0↓
BBSE first	30.5 ↓	27.7 ↓	30.5↓	39.2↓	30.5 ↓	28.1 ↓	30.6↓	40.3↓
Error-rate monitoring	52.3 ↓	48.5 ↓	52.2↓	62.8↓	52.5 ↓	49.6 ↓	53.1↓	64.8↓
Learned meta-classifier	97.5	31.1 ↓	95.8↑	48.2↓	98.5 ↑	39.2 ↓	97.4↑	51.2↓
Ours without MMD block	95.2	62.9	86.6	81.3	95.1	63.9	90.5	81.3
Ours, audit null without confidence	79.1 ↓	62.9	83.7↓	80.7	78.0 ↓	62.9	85.4↓	82.0
Ours, audit null with class cells	93.1	65.6 ↑	86.2	91.8↑	92.9 ↓	66.5 ↑	90.6	93.7↑

Inc. and Emp. stand for ACS Income and ACS Employment and A and B are the two benchmark designs. An arrow marks a method that differs significantly from ours after Holm correction: a down arrow means that ours is better and an up arrow that ours is worse. The upper block lists the competing methods and the lower block the ablations of our procedure.

With LR on ACS Income, our procedure names the type in 94.3% of design A windows (95% interval [92.3, 95.8]), while the best rule baseline reaches 52.3%. The rules are held back by design, since each can name only some of the five outcomes. Over all eight configurations our procedure

is significantly better in 28 of the 32 comparisons with a baseline or the learned rule and significantly worse in 3, all against the learned meta-classifier on design A (ACS Employment LR, ACS Income GB and ACS Employment GB).

On design A the learned meta-classifier is the strongest competitor. Validated on states it has not seen, it reaches 97.5% on ACS Income, above our 94.3%. The held-out mechanisms of design B turn this around. Trained on design A, the meta-classifier drops to 31.1% on ACS Income and 48.2% on ACS Employment, while our procedure keeps 62.4% and 80.5%. The learned rule had tied each type to the size and shape of the injected shift; it had learned, for example, that covariate shift means a large shift in two features and so it missed selection bias, which moves many features a little. This is the main argument for tests over a learned rule: the tests encode what the drift types are, not what the training examples happened to look like. Fig. 7.12 breaks the results down by type for LR.

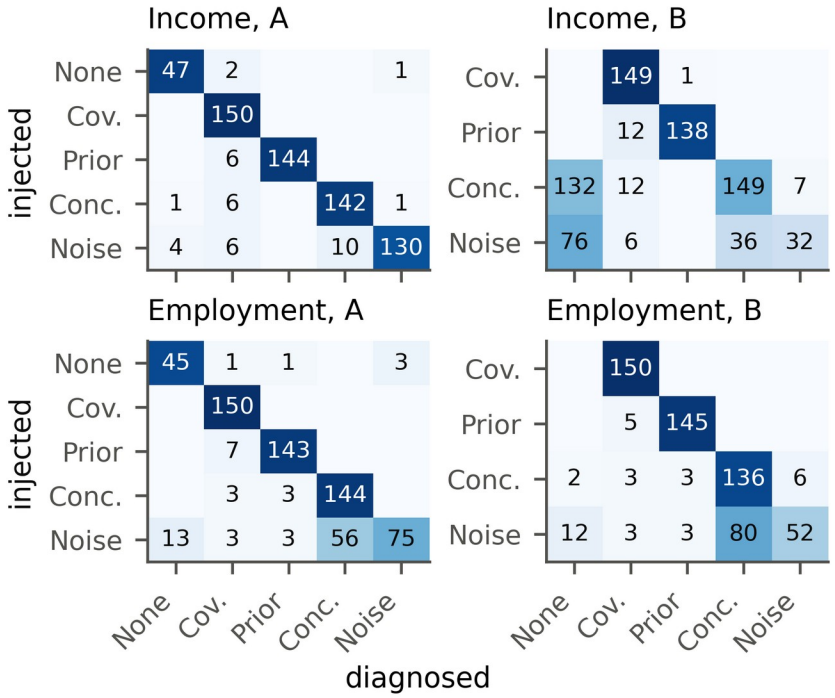


Fig. 7.12: Injected (rows) and diagnosed (columns) drift types on the benchmark with LR, in counts of windows; darker cells hold a larger share of their row

A perfect diagnosis would fill only the diagonal. Design B has no control windows, so false alarms are measured on design A, where most errors sit in the label noise row: noise is read either as concept drift or as no drift. Our procedure raised a false alarm on 13 of 200 control windows (6.5%, 95% CI [3.8, 10.8]), 7 in the label-free stage and 6 in the audit, below the bound of 10%.

The ablations show which parts of the procedure carry the weight. Without the confidence term in the audit null, accuracy on ACS Income design A falls by 15.2 points, because uniform label noise then looks like concept drift. Dropping the MMD block changes little on the benchmark (−0.9 points), whose shifts are visible feature by feature; its value shows on the image embeddings of CLEAR-10. Replacing the confidence bins by cells of predicted class and confidence helps on ACS Employment design B (91.8% against 80.5%) but costs accuracy on ACS Income design A (93.1%). Switching to it after seeing design B would bias the comparison, so it stays an ablation and a direction for future work.

7.4.2 Held-out Mechanisms

Design B asks whether the procedure carries over to drift it was not built for. Table 7.3 gives the result per mechanism.

Table 7.3: Correct-type rate (Corr., %) and mean accuracy drop (Drop, points) per held-out mechanism of design B

	ACS Income				ACS Employment			
	LR		GB		LR		GB	
Mechanism	**Corr.**	**Drop**	**Corr.**	**Drop**	**Corr.**	**Drop**	**Corr.**	**Drop**
Covariate: selection bias	99	0.2	99	0.3	100	−0.3	100	−0.3
Prior: decrease	92	−2.8	93	−2.4	97	2.5	97	1.9
Concept: oblique region	95	21.9	95	22.5	91	15.7	97	17.7
Concept: income threshold	4	−0.3	6	0.2	–	–	–	–
Noise: class-conditional	21	2.8	23	3.1	35	7.7	33	8.0

Three of the five mechanisms carry over well. Selection bias is named covariate shift in 99% of ACS Income windows with LR, even though no value is distorted and the shift is spread across all features. A lower positive rate is named prior shift in 92% and concept drift in an oblique region, which no single feature bounds, is still recognized in 95%.

The other two are harder. A higher income threshold relabels some rows, yet model accuracy hardly moves (mean drop −0.3 points), so the audit stops at its first question and reports no drift in 132 of the 150 windows. This is intended: the procedure looks for drift that costs accuracy and this drift costs almost none. Class-conditional label noise is recognized in only 21% of ACS Income windows. At low flip rates the loss is too small to show in 600 labels and at high rates the errors follow the true class, which is itself correlated with the features, so the error-structure test

reads the pattern as concept drift. A null model built on confidence alone cannot separate the two, a true limitation of the method.

7.4.3 Size of the Delayed Audit

Labels cost money, so it matters how many the audit needs. We varied the audit size on ACS Income design A with LR. Fig. 7.13 shows the result.

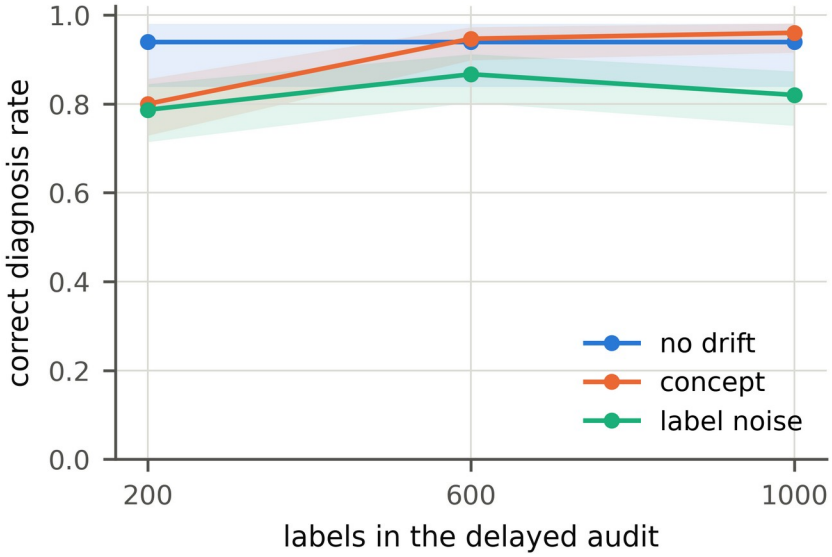


Fig. 7.13: Share of concept drift and label noise windows identified correctly at audit sizes of 200, 600 and 1000 labels (ACS Income, design A, LR), with 95% intervals

With 200 labels concept drift is identified in 80% of windows and label noise in 79%. With 600 labels the two rates reach 95% and 87% and going on to 1,000 labels adds little (96% and 82%). The 600 labels used throughout therefore sit where more labels stop paying off.

7.4.4 Detection and Accuracy Estimation

The diagnosis rests on the detectors and on the accuracy estimate, so both were checked on the benchmark. Table 7.4 gives the alarm rate of every detector by injected type.

Table 7.4: Alarm rate of each detector by injected type on design A (%), with the 95% interval on the control windows

	Alarm rate (%) by injected type				
Detector	**No drift [95% CI]**	**Covariate**	**Prior**	**Concept**	**Noise**
KS	1.0 [0.3, 3.6]	100.0	100.0	1.0	1.0
PSI	0.0 [0.0, 1.9]	99.7	0.3	0.0	0.0
MMD	7.0 [4.2, 11.4]	100.0	100.0	7.0	7.0

	Alarm rate (%) by injected type				
Detector	**No drift [95% CI]**	**Covariate**	**Prior**	**Concept**	**Noise**
Confidence	5.0 [2.7, 9.0]	93.8	86.7	5.0	5.0
BBSD	2.5 [1.1, 5.7]	100.0	100.0	2.5	2.5
Domain classifier	6.0 [3.5, 10.2]	100.0	72.0	6.0	6.0

On the 200 control windows the 95% interval of every detector's false alarm rate contains $\alpha = 0.05$ or lies below it. All detectors catch covariate shift and all except PSI catch most prior shifts. In concept drift and label noise windows the inputs are the same as in the control window of their replicate, so no detector can see them; their alarm rates equal the control rates. This is why the audit is needed. Table 7.5 compares the four accuracy estimates.

Table 7.5: Mean absolute error of the four accuracy estimates (percentage points); the best value in each column is in bold

	Benchmark, design A				Real windows			
Estimator	**No drift**	**Covariate**	**Prior**	**Concept, noise**	**ACS Inc.**	**ACS Emp.**	**Gas**	**CLEAR**
CBPE	**0.50**	**9.20**	3.15	14.71	1.86	1.06	30.98	0.82
AC	0.83	9.58	3.65	**14.66**	1.87	1.25	30.36	5.32
DoC	**0.50**	9.63	3.12	14.71	1.77	1.06	29.78	0.76
ATC	0.63	10.31	**2.43**	**14.66**	**1.15**	**1.02**	**26.95**	**0.62**

On design A all four estimates stay within 0.83 points without drift and within 10.31 points under covariate shift. Under concept drift and label noise every error equals the whole loss, about 14.71 points, because the inputs do not change. Across the 8 columns ATC has the smallest error in 6 and CBPE in 2. On the gas sensor data all four miss by 26.95 to 30.98 points on average: in batch 8 the mean confidence of the LR model falls from 0.973 to 0.595 and its accuracy to 0.330, yet CBPE estimates 0.618. The estimate is best read as an upper bound on accuracy.

7.4.5 Real Drift

On real data the type of drift is not known, but its causes are on record. The question is whether the verdicts agree with those causes and whether the control windows stay quiet. Fig. 7.14 shows how accuracy changes over time in all four datasets.

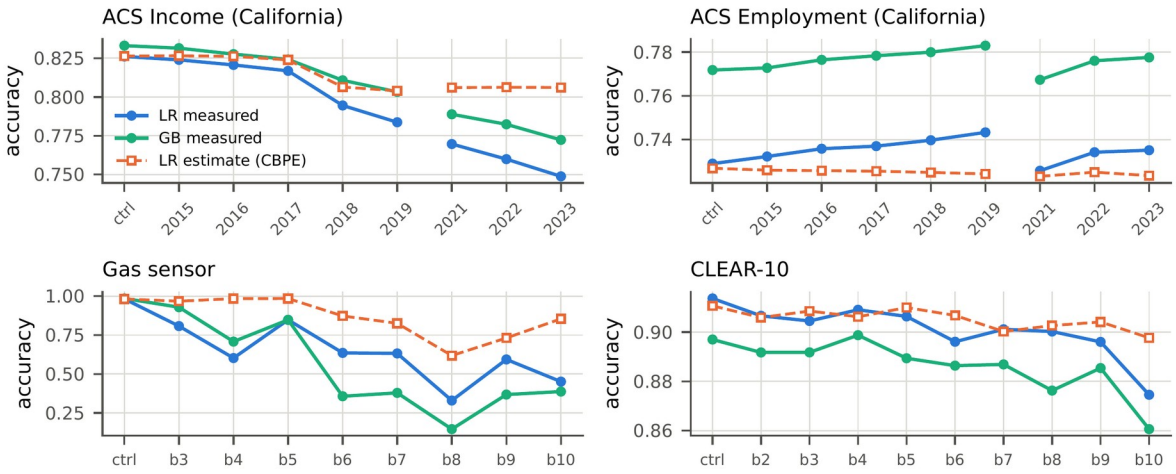


Fig. 7.14: Measured accuracy of both models and the label-free CBPE estimate on the temporal windows of the four datasets

C marks the control window and the numbers mark years (ACS), batches (gas sensor) or buckets (CLEAR-10). The gap in the ACS lines is the missing 2020 release. Across the four datasets no real window was named prior shift, concept drift or label noise. Most of the documented causes act on the inputs, from new codes and a redesigned question to aging sensors, so covariate verdicts are what one would expect. Table 7.6 gives the details for ACS Income, which shows the clearest trend.

Table 7.6: Detection and diagnosis on the ACS Income windows. The alarms are those of the LR model; Conf. is the confidence test and DC the domain classifier

	Accuracy				Alarms (LR)						Type	
Window	**LR**	**ΔLR**	**ΔGB**	**Est. err.**	**KS**	**MMD**	**Conf.**	**BBSD**	**DC**	**Top shifts**	**LR**	**GB**
CA 2014 (control)	0.826	0.0	0.0	0.0	–	–	–	–	–	–	–	–
CA 2015	0.824	−0.2	−0.2	+0.3	√	–	–	–	–	OCCP	–	–
CA 2016	0.821	−0.5	−0.5	+0.6	√	–	–	√	√	OCCP, POBP	Cov.	Cov.
CA 2017	0.817	−0.9	−0.9	+0.7	√	√	√	√	–	OCCP, POBP	Cov.	Cov.
CA 2018	0.795	−3.2	−2.2	+1.2	√	√	√	√	√	OCCP, POBP	Cov.	Cov.
CA 2019	0.784	−4.2	−3.0	+2.0	√	√	√	√	√	OCCP, RELP	Cov.	Cov.
CA 2021	0.770	−5.6	−4.4	+3.6	√	√	√	√	√	OCCP, RAC1P	Cov.	Cov.
CA 2022	0.760	−6.6	−5.1	+4.6	√	√	√	√	√	OCCP, RAC1P	Cov.	Cov.
CA 2023	0.749	−7.7	−6.1	+5.7	√	√	√	√	√	OCCP, RAC1P	Cov.	Cov.
TX 2014	0.802	−2.4	−2.7	+1.1	√	√	√	√	√	POBP, RAC1P	Cov.	Cov.

	Accuracy				Alarms (LR)						Type	
Window	**LR**	**ΔLR**	**ΔGB**	**Est. err.**	**KS**	**MMD**	**Conf.**	**BBSD**	**DC**	**Top shifts**	**LR**	**GB**
NY 2014	0.804	−2.2	−2.7	+0.8	√	√	√	√	√	POBP, RAC1P	Cov.	Cov.
FL 2014	0.786	−4.0	−4.3	+2.2	√	√	√	√	√	POBP, RAC1P	Cov.	Cov.
WA 2014	0.801	−2.5	−2.5	+0.9	√	√	√	√	√	POBP, RAC1P	Cov.	Cov.

Δ is the change in accuracy, in points, against each model's accuracy on the control window and Est. err. is the error of the CBPE estimate. A tick marks an alarm and Type is the verdict of the diagnosis, the same for LR and GB in every window. Top shifts lists the two features with the largest effect size.

Accuracy falls over time for both models, with LR going from 0.824 in 2015 to 0.749 in 2023, while the CBPE estimate falls far less. Neither model raises an alarm on the control window; both call CA 2015, whose accuracy did not change significantly, no drift and every other window covariate shift. The documented events can be read off the localization. The 2018 revision of the occupation codes left 26.6% of rows with a code that never occurred in 2014 and occupation lost weight in the model as a result (SHAP change −0.032). Relationship moves in 2019 (effect size 0.091), when its survey question was redesigned and race moves from 2021 (effect size 0.186), once the Census Bureau had changed its coding of written race answers [55]. Wage growth against a fixed income threshold shows up in the labels instead: their positive share rises from 0.355 in the control window to 0.498 in 2023 while the model predicts only 0.338. Part of the 2023 loss therefore comes from a change in what the labels mean, which no label-free test can detect.

The cost of a drift differs widely between datasets. On ACS Employment every window is diagnosed as covariate shift, yet LR accuracy never changes by more than 2.2 points; an alarm alone would overstate the problem and the accuracy estimate keeps it in proportion. The gas sensor batches all come out as covariate shift, as they should for aging sensors that alter $P(X \mid Y)$, with the largest loss in batch 8 (−65.2 points for LR). BBSE at first takes each gas batch for a prior shift, since the predicted labels move too, but the consistency test corrects this: the sensors respond differently to a gas mix that has not changed. CLEAR-10 drifts more gently. Buckets 2 and 3 show no drift for either model and buckets 4, 5, 6, 7, 8, 9 and 10 show covariate shift. In buckets 4, 5, 6 and 9 no single embedding dimension shifts significantly with LR; only the MMD test sees the change, which is why the multivariate block is needed. The only detector alarm on any control

window comes from the domain classifier baseline on CLEAR-10 ($p = 0.034$), a false alarm of the kind that α allows.

7.4.6 Reliability and Triage

The fused risk score is judged by how well it ranks wrong predictions above right ones. Where the fusion is learned, on ACS and CLEAR-10, its AUROC differs from that of confidence alone by −0.012 to +0.001 over 72 windows and by −0.002 to +0.009 on the benchmark. On the gas sensor array the calibration part holds too few wrong predictions (4 with LR and 11 with GB), so the risk falls back to confidence. Confidence alone carries most of the information about a single prediction; the other signals add almost nothing, an honest negative result.

For the triage, a retrain truly helps when importance weighting or the latest labeled window raises holdout accuracy significantly. Table 7.7 gives the outcome per setting.

Table 7.7: Remediation by setting: settings with a significant gain from importance weighting (IW), retraining on the recent window and retraining on fresh labels and the triage's advice

			Significant gains			Triage		
Model	**Settings**	*n*	**IW**	**Recent**	**Labels**	**Escalated**	**Truly helps**	**Correct**
LR	ACS Income, temporal	8	4/8	7/7	5/8	7	7	8/8
	ACS Income, states	4	0/4	n/a	4/4	0	0	4/4
	ACS Employment, temporal	8	4/8	7/7	2/8	7	7	8/8
	ACS Employment, states	4	2/4	n/a	1/4	1	2	3/4
	CLEAR-10	9	0/9	4/8	6/9	4	4	7/9
	Gas sensor	8	1/8	7/7	8/8	7	7	8/8
GB	ACS Income, temporal	8	0/8	7/7	4/8	7	7	8/8
	ACS Income, states	4	0/4	n/a	3/4	0	0	4/4
	ACS Employment, temporal	8	0/8	7/7	0/8	6	7	7/8
	ACS Employment, states	4	0/4	n/a	0/4	0	0	4/4
	CLEAR-10	9	0/9	8/8	9/9	8	8	9/9
	Gas sensor	8	1/8	5/7	8/8	7	6	7/8

Over 82 real settings the triage is right in 77. It is right in 54 of the 58 settings that have an earlier labeled window, because its recent-window probe is checked on real labels. The geographic

windows of the census tasks have no earlier window; there, importance weighting rarely helps and the triage is right in 23 of 24 settings, but mostly by advising against a retrain, since one truly helped in only 2 of them. On the benchmark its recall is only 6%, because importance-weighted validation cannot see gains outside the reference data.

7.4.7 Computational Cost

A monitor has to finish with one window before the next one arrives, so the time of each window was recorded through the stages of the pipeline, from the detectors to the reliability scores, leaving out model training and the image embeddings. Over 90 real windows no window took longer than 90 seconds and all of them together took 45.5 minutes. With LR on the census tasks most of the time goes to the SHAP values of the root cause analysis; on CLEAR-10 and the gas sensor array the diagnosis takes the largest share. The whole injection benchmark took 1.6 hours on 20 processor threads. Windows of a day, a week or a month leave ample room; a stream that needs a verdict every few seconds would call for cheaper tests.

7.5 Comparison of Results with Existing Systems

The comparison in Table 7.2 is fair in three respects. Every method names one of the same five outcomes, from the same tests and the same audit labels, so differences come from how the evidence is turned into a verdict, not from different data. The rule baselines stand for current practice: input monitoring reads any input alarm as covariate shift, BBSE first reads a shift in the predicted labels as prior shift [3], [11] and error-rate monitoring also reads a drop in audit accuracy as concept drift [7]. The learned meta-classifier, a GB model trained on nine statistics of each window from the same tests, checks whether a learned rule could replace the tests.

Against the rules, our procedure is better in every configuration, by a wide margin, because each rule can name only some of the types. Against the learned rule, the picture depends on whether the drift was seen in training: the meta-classifier wins narrowly on design A and loses by a wide margin on every configuration of design B. A monitoring team does not know in advance which drift it will meet, so a procedure that holds up on unseen mechanisms is the safer choice. Compared with the published methods of Table 2.4, the system is the only one that separates concept drift from label noise and the only one evaluated on held-out drift. Its accuracy estimates are on par with the best published estimators: ATC is slightly more accurate in most columns of Table 7.5 and all four fail together when confidence stops tracking accuracy.

7.6 Inference Drawn

The results support four conclusions. First, hypothesis tests with a stated error rate can name the drift type from label-free tests and a few hundred late labels: 85.7 to 94.3% on design A and 62.4 to 81.5% on held-out drift, with false alarms below their bound. The project's objectives for the diagnosis were met. Second, encoding the definitions of the types makes the diagnosis carry over to new drift, where a learned rule does not. Third, on real data the system stays quiet on control windows and traces documented events, from new occupation codes to aging sensors, while showing that an alarm alone can overstate a harmless shift. Fourth, the supporting stages have clear limits: the accuracy estimate is an upper bound, the reliability fusion adds little beyond confidence and the triage is reliable only where an earlier labeled window exists.

Two results were unexpected. The first was how strongly the meta-classifier collapsed on design B, which made the case for tests clearer than we had anticipated. The second was class-conditional label noise, which the audit mostly reads as concept drift; this exposed a real limit of a confidence-based null and points to the main next step of the work.

7.7 Sustainability Impact Assessment and Rubric Justification

This section assesses the sustainability of the project with the structure the department provides: its scope and evidence, the material topics, targets and progress, the link to the Sustainable Development Goals, a self-assessment against the rubric and a statement of limitations. It distinguishes throughout between what was measured, what was estimated and what was not assessed.

7.7.1 Scope, Boundary and Evidence

The assessment covers the development and evaluation of ResilientAI from February to October 2026: the experiments on the team's workstation, the dashboard and the documents. Hosting, use by a real monitoring team, maintenance and disposal are outside the boundary, because the system has not been deployed. The users considered are the stakeholders of Section 3.1. The evidence consists of the measured run times stored in every study report, the benchmark's recorded wall-clock time, the processor's rated maximum power of 181 W [60], the average emission factor of the Indian grid of 0.727 kg of carbon dioxide per kWh for 2023–24 [61] and the design documents of the system. There is no earlier version of the system to serve as a baseline, so comparisons are made against the alternative a team would otherwise use, such as labeling every window.

7.7.2 Material Sustainability Topics

Table 7.8 lists the environmental, social and governance topics, their relevance to this project and the evidence for each.

Table 7.8: Material sustainability topics

Dimension / topic	Project relevance and potential impact	Indicator / unit	Evidence source	Boundary / limitation
Environment: energy / compute	Material: experiments and monitoring use electricity; the triage can avoid needless retraining	kWh; seconds per window	Stored run times; rated power	Upper-bound estimate; not metered; development time excluded
Environment: emissions / travel	Material through electricity use; no travel was needed beyond normal college attendance	kg CO_2	Energy × grid factor	Grid average, not marginal factor
Environment: materials / waste	Not material: no hardware was bought or discarded	–	Budget (Table 2.5)	Existing workstation's life cycle not assessed
Social: access / inclusion / usability	Monitoring teams of any size can run the system on ordinary hardware; dashboard explains each step in plain language	Steps with a question and a generated note	Dashboard (Section 7.1)	No study with outside users
Social: safety / well-being / community	Catching a failing model early protects the people its decisions affect	False alarm rate; correct-type rate	Section 7.4	Indirect; no deployment
Governance: privacy / security / fairness	Public data only, local processing; census tasks include sensitive attributes	Personal records collected; data leaving the machine	Design; dataset licences	No fairness audit of the monitored models

7.7.3 Sustainability Targets and Progress

Table 7.9 sets project-specific targets for the material topics. They were set for this assessment from the requirements of Chapter 3; where no baseline exists, the alternative a team would otherwise use serves as the reference.

Table 7.9: Sustainability targets and progress

Topic / target	Baseline and date	Target / due date	Actual result	Status / corrective action
Labels needed per window	Label the whole window: 4,000 rows (benchmark)	At most 600, by October 2026	600 audit labels, only when no label-free test rejects	Met
Retrain only when it helps	Retrain on every alarm	Triage right in most settings, by October 2026	Right in 77 of 82 real settings	Met with real labels; low recall without them; future work
Compute per window	None	Within a daily cycle on one workstation	At most 90 s per window	Met
Energy of all experiments	None	Below 5 kWh	About 0.43 kWh (upper bound)	Met; meter future runs
Personal data collected	None	Zero records	Zero	Met

Topic / target	Baseline and date	Target / due date	Actual result	Status / corrective action
Plain-language explanation	None	Every dashboard step	Every step has a question and a generated note	Met; no accessibility audit yet

7.7.4 SDG Alignment and Evidence

The links of Section 1.7 can now be graded. For SDG 9, target 9.5, the link is a tested output: the audit test and the error-controlled diagnosis were built and evaluated with the results above and then described in a research paper. For target 9.4 it is an expected contribution: the system keeps deployed models reliable in evaluation, but it has not yet been used in a real deployment. For SDG 12, target 12.2, the label economy is a tested output (600 labels instead of whole windows), while the computation saved by the triage is a design intent supported by its accuracy on real settings but not measured in a deployment. For target 12.6 the link is demonstrated by this section, which reports energy, emissions, privacy and inclusion with its method and limits. We do not claim any measurable effect on national or global progress towards these goals; the project is a small contribution to their targets.

7.7.5 Sustainability Assessment Rubric

The department's rubric rates six criteria from 0 to 3. Table 7.10 reproduces it and Table 7.11 gives our self-assessment, to be confirmed by the mentor and examiners.

Table 7.10: Sustainability assessment rubric of the department

Criterion	Weight	0 – Not evidenced	1 – Initial	2 – Developing	3 – Strong
A. Scope, materiality and baseline	15%	No boundary or relevant topics identified	Boundary/topics stated but vague; no baseline rationale	Boundary and material topics explained; baseline partly defined	Clear life-cycle boundary, justified material topics, reporting period and baseline/data gaps stated
B. Environmental/resource impacts	20%	No relevant impact considered	Mentions impacts without method or evidence	Uses relevant indicator(s) with method and some evidence	Quantifies/estimates relevant energy, emissions, materials/waste or explains N/A with sound rationale; method and limits disclosed
C. Social value, inclusion and safety	20%	No social effects considered	Benefits asserted without stakeholder evidence	Stakeholders and relevant usability/accessibility/safety measures identified	Impact on affected users assessed with suitable tests/feedback; inclusion, safety and limitations documented
D. Governance, privacy, security and ethics	20%	Risks not considered	Generic statements only	Relevant risks and controls documented; some verification	Data handling, consent/licensing, security, fairness/accountability controls and test evidence clearly documented
E. SDG linkage and measurable targets	10%	No SDG/target linkage	SDGs listed without causal explanation	Relevant SDG target and plausible indicator linked to project	Target-level mapping, baseline/target/indicator and supporting evidence; expected vs demonstrated impact distinguished

Criterion	Weight	0 – Not evidenced	1 – Initial	2 – Developing	3 – Strong
F. Measurement, transparency and improvement	15%	No results or limitations	Anecdotal results; no method or limitations	Metrics/methods/results partly reproducible; limitations noted	Evidence traceable and repeatable; assumptions, uncertainty, gaps and corrective/future actions disclosed

Table 7.11: Self-assessment against the rubric. Weighted points = (rating ÷ 3) × weight

Criterion	Weight	Rating	Points	Basis
A. Scope, materiality and baseline	15	2	10.0	Boundary, period and material topics stated; no measured baseline, only the alternative practice
B. Environmental/resource impacts	20	2	13.3	Energy and emissions estimated with a stated method and sources; not metered
C. Social value, inclusion and safety	20	2	13.3	Stakeholders and usability measures identified; no test with outside users
D. Governance, privacy, security and ethics	20	2	13.3	Licences, privacy and local processing documented; no fairness audit
E. SDG linkage and measurable targets	10	2	6.7	Target-level mapping with indicators; expected and demonstrated effects distinguished; few numeric baselines
F. Measurement, transparency and improvement	15	3	15.0	Every number generated from stored outputs and repeatable with seeds; assumptions and gaps disclosed
Total	100		71.7	Satisfactory (60–79) on the department's suggested scale

The strongest criterion is transparency, because every figure in the assessment can be traced to stored outputs and rebuilt. The weakest are the social and governance criteria, where the evidence stops at identification: a usability study with outside users and a fairness audit of the monitored models would raise both.

7.7.6 Evidence and Limitations Statement

The run times and the benchmark's wall-clock time were measured and are stored in the study reports. The energy and emission figures are estimated: they multiply measured time by the processor's rated maximum power, which overstates the real draw and by the average emission factor of the Indian grid for 2023–24 (later versions of the database give lower factors, so this also errs on the high side). They exclude the graphics card's work on image embeddings, the development time and the time spent writing. Inclusion and usability were assessed qualitatively by the team and in mentor reviews. Fairness and accessibility were not assessed. The savings in labels are tested on the benchmark; the savings in computation from the triage are not measured in a deployment. Future work should meter the energy of a full run, run a short usability study with a monitoring team and add a fairness check of the monitored models.

CHAPTER 8: CONCLUSION

This project set out to turn a drift alarm into a decision. ResilientAI watches a deployed classifier, tests every new window of production data for a change, estimates the accuracy the model is likely to have, locates the features that moved and names the type of drift, so that a monitoring team knows whether to reweight, recalibrate, collect fresh labels or check its labeling process. It also explains the change, scores the trust due to each prediction, advises whether a cheap retrain would help and presents everything in a dashboard.

The central result is that hypothesis tests with a stated error rate can name the drift type from label-free tests and a few hundred late labels. The new audit test separates concept drift from label noise by asking whether the change in errors depends on a feature beyond what the model's confidence explains. On injected drift the procedure named the type in 85.7 to 94.3% of windows, beat every rule from monitoring practice and kept its false alarms below their bound. Unlike a learned meta-classifier, it carried over to drift mechanisms held out from its design, because its tests encode the definitions of the types rather than the look of past examples. On four real datasets it raised no false verdict on any control window and traced documented events, from new occupation codes in the census to aging gas sensors.

The project met the deliverables of Section 2.7. Along the way it also showed the value of practices that are easy to skip in a student project: fixing the protocol before the evaluation, generating every reported number from stored outputs and testing the error rates of the statistics themselves. Those practices caught real errors and made it possible to correct them honestly.

8.1 Limitations

The system has the following limitations, each visible in the results of Chapter 7.

- **One type per window.** Each benchmark window holds a single drift type, while real windows such as ACS Income 2023 mix several. The procedure reports the first type it finds.
- **Idealized types.** Sensor aging changes $P(X \mid Y)$ and, through it, $P(Y \mid X)$, yet the procedure reports it as covariate shift, because the four types are an idealization of real change.
- **Class-conditional label noise.** The audit often mistakes noise that depends on the true class for concept drift, since a null model built on confidence alone cannot separate the two.

- **Drift without loss.** The audit lets through drift that does not hurt accuracy, such as the raised income threshold. This is by design but may surprise a user.
- **Sampling error of the weights.** The weighted residual tests do not account for the sampling error of the BBSE weights.
- **Large samples.** At several hundred thousand rows almost any shift is significant, so each decision must be read with its effect size.
- **Optimistic estimates.** The accuracy estimate fails when the model stays confident while its accuracy falls, as on the gas sensor data and is best read as an upper bound.
- **Weak supporting stages.** The reliability fusion adds almost nothing beyond confidence and the triage has low recall when no earlier labeled window exists.
- **Scope.** Deep models trained end to end, text data and real-time streams remain untested and the system has not been used by a monitoring team outside the project.

8.2 Future Scope

The limits above point to realistic next steps.

1. **An audit conditioned on the full model output**, such as cells of predicted class and confidence, to separate class-conditional label noise from concept drift. The ablation in Section 7.4 suggests this helps on held-out drift and it should be tested on a fresh benchmark design to avoid bias.
2. **A model of the sampling error of the BBSE weights** in the label shift consistency test.
3. **Benchmark windows that mix several types**, with a diagnosis that can report more than one.
4. **Text and deep models**, by applying the same tests to embeddings of text and to models fine-tuned on the production stream.
5. **Streaming operation**, with sequential tests that keep the error rate over an open-ended sequence of windows.
6. **A pilot deployment** with a monitoring team, together with a usability study, metered energy use and a fairness check of the monitored models, which would also complete the sustainability evidence of Section 7.7.
7. **Integration with existing monitoring platforms**, so that the diagnosis can run beside the detectors teams already use.

REFERENCES

[1] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera, “A Unifying View on Dataset Shift in Classification,” *Pattern Recognition*, vol. 45, no. 1, pp. 521–530, 2012, doi: 10.1016/j.patcog.2011.06.019.

[2] *Dataset Shift in Machine Learning*. MIT Press, 2009.

[3] S. Rabanser, S. Günnemann, and Z. C. Lipton, “Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift,” in *Advances in Neural Information Processing Systems 32*, 2019, pp. 1394–1406. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/hash/846c260d715e5b854ffad5f70a516c88-Abstract.html (accessed Oct. 21, 2026).

[4] D. Sculley et al., “Hidden Technical Debt in Machine Learning Systems,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 2503–2511.

[5] A. Paleyes, R.-G. Urma, and N. D. Lawrence, “Challenges in Deploying Machine Learning: A Survey of Case Studies,” *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–29, 2022, doi: 10.1145/3533378.

[6] C. Huyen, *Designing Machine Learning Systems*. O'Reilly Media, 2022.

[7] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, “Learning with Drift Detection,” in *Advances in Artificial Intelligence – SBIA 2004*, 2004, pp. 286–295, doi: 10.1007/978-3-540-28645-5_29.

[8] A. Bifet and R. Gavaldà, “Learning from Time-Changing Data with Adaptive Windowing,” in *Proceedings of the SIAM International Conference on Data Mining (SDM)*, 2007, pp. 443–448, doi: 10.1137/1.9781611972771.42.

[9] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, “A Kernel Two-Sample Test,” *Journal of Machine Learning Research*, vol. 13, pp. 723–773, 2012. [Online]. Available: https://jmlr.org/papers/v13/gretton12a.html (accessed Oct. 21, 2026).

[10] D. Lopez-Paz and M. Oquab, “Revisiting Classifier Two-Sample Tests,” in *5th International Conference on Learning Representations*, 2017. [Online]. Available: https://arxiv.org/abs/1610.06545 (accessed Oct. 21, 2026).

[11] Z. C. Lipton, Y.-X. Wang, and A. J. Smola, “Detecting and Correcting for Label Shift with Black Box Predictors,” in *Proceedings of the 35th International Conference on Machine Learning*, 2018, pp. 3122–3130. [Online]. Available: https://proceedings.mlr.press/v80/lipton18a.html (accessed Oct. 21, 2026).

[12] J. Kivimäki, J. Białek, W. Kuberski, and J. K. Nurminen, “Performance Estimation in Binary Classification Using Calibrated Confidence,” *Machine Learning*, vol. 115, no. 3, Art. no. 67, 2026, doi: 10.1007/s10994-025-06970-3.

[13] NannyML, “Confidence-Based Performance Estimation (CBPE),” 2023.

[14] J. Klaise, A. Van Looveren, C. Cox, G. Vacanti, and A. Coca, “Monitoring and Explainability of Models in Production,” in *ICML Workshop on Challenges in Deploying and Monitoring Machine Learning Systems*, 2020.

[15] F. Maia Polo, R. Izbicki, E. G. Lacerda, J. P. Ibieta-Jimenez, and R. Vicente, “A Unified Framework for Dataset Shift Diagnostics,” *Information Sciences*, vol. 649, Art. no. 119612, 2023, doi: 10.1016/j.ins.2023.119612.

[16] Y. Ovadia et al., “Can You Trust Your Model's Uncertainty? Evaluating Predictive Uncertainty under Dataset Shift,” in *Advances in Neural Information Processing Systems 32*, 2019, pp. 13991–14002. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/hash/8558cb408c1d76621371888657d2eb1d-Abstract.html (accessed Oct. 21, 2026).

[17] United Nations General Assembly, “Transforming our world: the 2030 Agenda for Sustainable Development, Resolution A/RES/70/1,” 2015. [Online]. Available: https://sdgs.un.org/2030agenda (accessed Oct. 30, 2026).

[18] J. Gama, Žliobait\.e, Indr\.e, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A Survey on Concept Drift Adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014, doi: 10.1145/2523813.

[19] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under Concept Drift: A Review,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2019, doi: 10.1109/TKDE.2018.2876857.

[20] G. I. Webb, R. Hyde, H. Cao, H. L. Nguyen, and F. Petitjean, “Characterizing Concept Drift,” *Data Mining and Knowledge Discovery*, vol. 30, no. 4, pp. 964–994, 2016, doi: 10.1007/s10618-015-0448-4.

[21] M. Roschewitz, R. Mehta, C. Jones, and B. Glocker, “Automatic Dataset Shift Identification to Support Safe Deployment of Medical Imaging AI,” in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2025*, 2025, pp. 67–76, doi: 10.1007/978-3-032-04981-0_7.

[22] D. Hendrycks and K. Gimpel, “A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks,” in *5th International Conference on Learning Representations*, 2017. [Online]. Available: https://arxiv.org/abs/1610.02136 (accessed Oct. 21, 2026).

[23] D. Guillory, V. Shankar, S. Ebrahimi, T. Darrell, and L. Schmidt, “Predicting with Confidence on Unseen Distributions,” in *2021 IEEE/CVF International Conference on Computer Vision*, 2021, pp. 1114–1124, doi: 10.1109/ICCV48922.2021.00117.

[24] S. Garg, S. Balakrishnan, Z. C. Lipton, B. Neyshabur, and H. Sedghi, “Leveraging Unlabeled Data to Predict Out-of-Distribution Performance,” in *The Tenth International Conference on Learning Representations*, 2022. [Online]. Available: https://arxiv.org/abs/2201.04234 (accessed Oct. 21, 2026).

[25] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html (accessed Oct. 21, 2026).

[26] F. Ding, M. Hardt, J. Miller, and L. Schmidt, “Retiring Adult: New Datasets for Fair Machine Learning,” in *Advances in Neural Information Processing Systems 34*, 2021, pp. 6478–6490. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2021/hash/32e54441e6382a7fbacbbbaf3c450059-Abstract.html (accessed Oct. 21, 2026).

[27] Z. Lin, J. Shi, D. Pathak, and D. Ramanan, “The CLEAR Benchmark: Continual LEArning on Real-World Imagery,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1*, 2021. [Online]. Available: https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/2838023a778dfaecdc212708f721b788-Abstract-round2.html (accessed Oct. 21, 2026).

[28] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, “Learning Deep Features for Discriminative Localization,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 2921–2929, doi: 10.1109/CVPR.2016.319.

[29] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *2008 Eighth IEEE International Conference on Data Mining*, 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.

[30] P. W. Koh et al., “WILDS: a benchmark of in-the-wild distribution shifts,” in *Proceedings of the 38th International Conference on Machine Learning*, 2021, pp. 5637–5664.

[31] J. Gardner, Z. Popovic, and L. Schmidt, “Benchmarking distribution shift in tabular data with TableShift,” in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023.

[32] J. Liu, T. Wang, P. Cui, and H. Namkoong, “On the need for a language describing distribution shifts: illustrations on tabular datasets,” in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023.

[33] E. A. Fly and T. McDaniel, “Real-time drift detection in machine learning systems and applications,” U.S. Patent 10 762 444 B2, Sep. 1, 2020. [Online]. Available: https://patents.google.com/patent/US10762444B2/en (accessed Oct. 30, 2026).

[34] K. E. Hines, J. P. Dickerson, K. Rao, R. Cheung, and R. M. E. Hyde, “Systems and methods for detecting drift between data used to train a machine learning model and data used to execute the machine learning model,” U.S. Patent 11 568 167 B2, Jan. 31, 2023. [Online]. Available: https://patents.google.com/patent/US11568167B2/en (accessed Oct. 30, 2026).

[35] V. K. Kolar, J.-P. Vasseur, G. Mermoud, and P.-A. Savalle, “Anomaly detection of model performance in an MLOps platform,” U.S. Patent 12 199 847 B2, Jan. 14, 2025. [Online]. Available: https://patents.google.com/patent/US12199847B2/en (accessed Oct. 30, 2026).

[36] U. K. Gosala, R. Prakash, G. Cherian, and R. Velega, “Systems and methods for detecting drift,” U.S. Patent 12 282 384 B2, Apr. 22, 2025. [Online]. Available: https://patents.google.com/patent/US20240303148A1/en (accessed Oct. 30, 2026).

[37] L. Kröger and E. Schwulera, “Drift detection for machine learning operations,” European Patent Appl. EP 4 459 516 A1, Nov. 6, 2024. [Online]. Available: https://patents.google.com/patent/EP4459516A1/en (accessed Oct. 30, 2026).

[38] Y. Zhang and W. Lin, “Real time detection, prediction and remediation of machine learning model drift in asset hierarchy based on time-series data,” European Patent Appl. EP 4 577 950 A1, Jul. 2, 2025. [Online]. Available: https://patents.google.com/patent/EP4577950A1/en (accessed Oct. 30, 2026).

[39] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html (accessed Oct. 21, 2026).

[40] P. Virtanen et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[41] C. R. Harris et al., “Array Programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[42] S. Holm, “A Simple Sequentially Rejective Multiple Test Procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: https://www.jstor.org/stable/4615733 (accessed Oct. 21, 2026).

[43] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in *Advances in Neural Information Processing Systems 30*, 2017, pp. 3146–3154. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html (accessed Oct. 21, 2026).

[44] N. Siddiqi, *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. Hoboken, NJ, USA: Wiley, 2006, doi: 10.1002/9781119201731.

[45] B. Phipson and G. K. Smyth, “Permutation P-values Should Never Be Zero: Calculating Exact P-values When Permutations Are Randomly Drawn,” *Statistical Applications in Genetics and Molecular Biology*, vol. 9, no. 1, Art. no. 39, 2010, doi: 10.2202/1544-6115.1585.

[46] W. G. Cochran, “Some Methods for Strengthening the Common χ^2 Tests,” *Biometrics*, vol. 10, no. 4, pp. 417–451, 1954, doi: 10.2307/3001616.

[47] L. Kish, *Survey Sampling*. New York, NY, USA: Wiley, 1965.

[48] P. Peduzzi, J. Concato, E. Kemper, T. R. Holford, and A. R. Feinstein, "A Simulation Study of the Number of Events per Variable in Logistic Regression Analysis," *Journal of Clinical Epidemiology*, vol. 49, no. 12, pp. 1373–1379, 1996, doi: 10.1016/S0895-4356(96)00236-3.

[49] C. Cortes, Y. Mansour, and M. Mohri, "Learning Bounds for Importance Weighting," in *Advances in Neural Information Processing Systems 23*, 2010, pp. 442–450. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2010/hash/59c33016884a62116be975a9bb8257e3-Abstract.html (accessed Oct. 21, 2026).

[50] H. Shimodaira, "Improving Predictive Inference under Covariate Shift by Weighting the Log-Likelihood Function," *Journal of Statistical Planning and Inference*, vol. 90, no. 2, pp. 227–244, 2000, doi: 10.1016/S0378-3758(00)00115-4.

[51] M. Sugiyama, M. Krauledat, and K.-R. Müller, "Covariate Shift Adaptation by Importance Weighted Cross Validation," *Journal of Machine Learning Research*, vol. 8, pp. 985–1005, 2007. [Online]. Available: https://jmlr.org/papers/v8/sugiyama07a.html (accessed Oct. 21, 2026).

[52] U.S. Census Bureau, "2020 ACS 1-Year Experimental Data Release," 2021. [Online]. Available: https://www.census.gov/programs-surveys/acs/data/experimental-data.html (accessed Oct. 21, 2026).

[53] U.S. Census Bureau, "American Community Survey 2019 PUMS Data Dictionary," 2020. [Online]. Available: https://www2.census.gov/programs-surveys/acs/tech_docs/pums/data_dict/PUMS_Data_Dictionary_2019.csv (accessed Oct. 21, 2026).

[54] J. B. Beckhusen, "Recent Changes in the Census Industry and Occupation Classification Systems," U.S. Census Bureau, Tech. Rep. ACS-TP78, 2020. [Online]. Available: https://www.census.gov/library/publications/2020/demo/acs-tp78.html (accessed Oct. 21, 2026).

[55] U.S. Census Bureau, "Improvements to the Race Question (ACS User Note 2021-03)," 2021. [Online]. Available: https://www.census.gov/programs-surveys/acs/technical-documentation/user-notes/2021-03.html (accessed Oct. 21, 2026).

[56] A. Vergara, S. Vembu, T. Ayhan, M. A. Ryan, M. L. Homer, and R. Huerta, "Chemical Gas Sensor Drift Compensation Using Classifier Ensembles," *Sensors and Actuators B: Chemical*, vol. 166--167, pp. 320–329, 2012, doi: 10.1016/j.snb.2012.01.074.

[57] I. Rodriguez-Lujan, J. Fonollosa, A. Vergara, M. Homer, and R. Huerta, "On the Calibration of Sensor Arrays for Pattern Recognition Using the Minimal Number of Experiments," *Chemometrics and Intelligent Laboratory Systems*, vol. 130, pp. 123–134, 2014, doi: 10.1016/j.chemolab.2013.10.012.

[58] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.

[59] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927, doi: 10.1080/01621459.1927.10502953.

[60] Intel Corporation, “Intel Core i5 processor 14600K (24M cache, up to 5.30 GHz): product specifications,” 2023. [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/236799/intel-core-i5-processor-14600k-24m-cache-up-to-5-30-ghz/specifications.html (accessed Oct. 30, 2026).

[61] Central Electricity Authority, “CO2 Baseline Database for the Indian Power Sector, User Guide Version 20.0, Table S,” 2024. [Online]. Available: https://cea.nic.in/cdm-co2-baseline-database/?lang=en (accessed Oct. 30, 2026).

APPENDIX – I: PROJECT EVIDENCE AND SUPPORTING DOCUMENTS

This appendix collects the evidence that the department asks to be attached: publication and intellectual property records, competition certificates, the progress review sheets, supporting material and the video presentation. Items that are issued or signed by others are inserted as scans in the places marked below.

A. IPR / Research Publication Evidence

The method and results of this project are described in a research paper written by the project team:

R. Guhagarkar, N. Cherian, A. Pal, A. Patre and S. Mirchandani, “Drift Detection, Diagnosis and Reliability Analysis for Deployed Machine Learning Models,” venue, submission date and status: submitted / accepted / presented.

- Research paper submitted, accepted or presented, with full citation and status evidence: insert the submission or acceptance record.
- Certificate of publication or presentation: insert if issued; otherwise write “Not yet issued”.
- Patent or other IPR filing: no patent has been filed for this project. The patent search of Section 2.3 found no granted or published patent that claims its drift type diagnosis.

B. Project Competition / Award Certificates

Attach certificates for project competitions or awards, if any. If not applicable, write “Not applicable.”

C. Progress Review Sheets

Attach Progress Review Sheet 1 and Progress Review Sheet 2 in the prescribed format.

D. Additional Supporting Material

Plagiarism and AI-content detection reports. Attach the plagiarism report and the AI-content detection report generated on the final text of this report.

Automated test run. The test suite was run on 30 October 2026 with pytest 9.0.2 under Python 3.14.8: 113 tests passed in 49 seconds (94 unit, 13 integration and 6 smoke tests), with no failures. The full log is kept with the project files (evidence/pytest-2026-10-30.txt).

How to run the system. From the root of the code repository, with Python 3.14 and the dependencies of pyproject.toml installed:

Algorithm 8.1: Commands to reproduce the studies and open the dashboard

```
python examples/quickstart/folktables_demo.py –task income –model lr
python examples/quickstart/gas_drift_study.py –model lr
python examples/quickstart/clear10_drift_study.py –model lr
python examples/quickstart/drift_injection_benchmark.py –dataset acs_income –
design A –model lr
python examples/quickstart/remediation_demo.py –model lr
python examples/quickstart/benchmark_summary.py
python examples/paper/build_artifacts.py
streamlit run examples/dashboard/app.py –server.address 127.0.0.1
```

Each study writes its report to the outputs folder; the build script regenerates the result values, tables and figures; the last command opens the dashboard of Section 7.1 in a browser on the local machine. Algorithm 8.1 lists the commands for one model; the other model family is run with `--model hgb`.

E. Video Presentation

Insert the video presentation folder link shared by the department and check that it opens for authorized reviewers.